# Unidad II · Actividad 3 · Gestión de roles y permisos

Sistema General de Ingresos — Evidencias de funcionamiento

Python · Strawberry GraphQL · SQLAlchemy · MySQL 8.4 (Docker Compose)

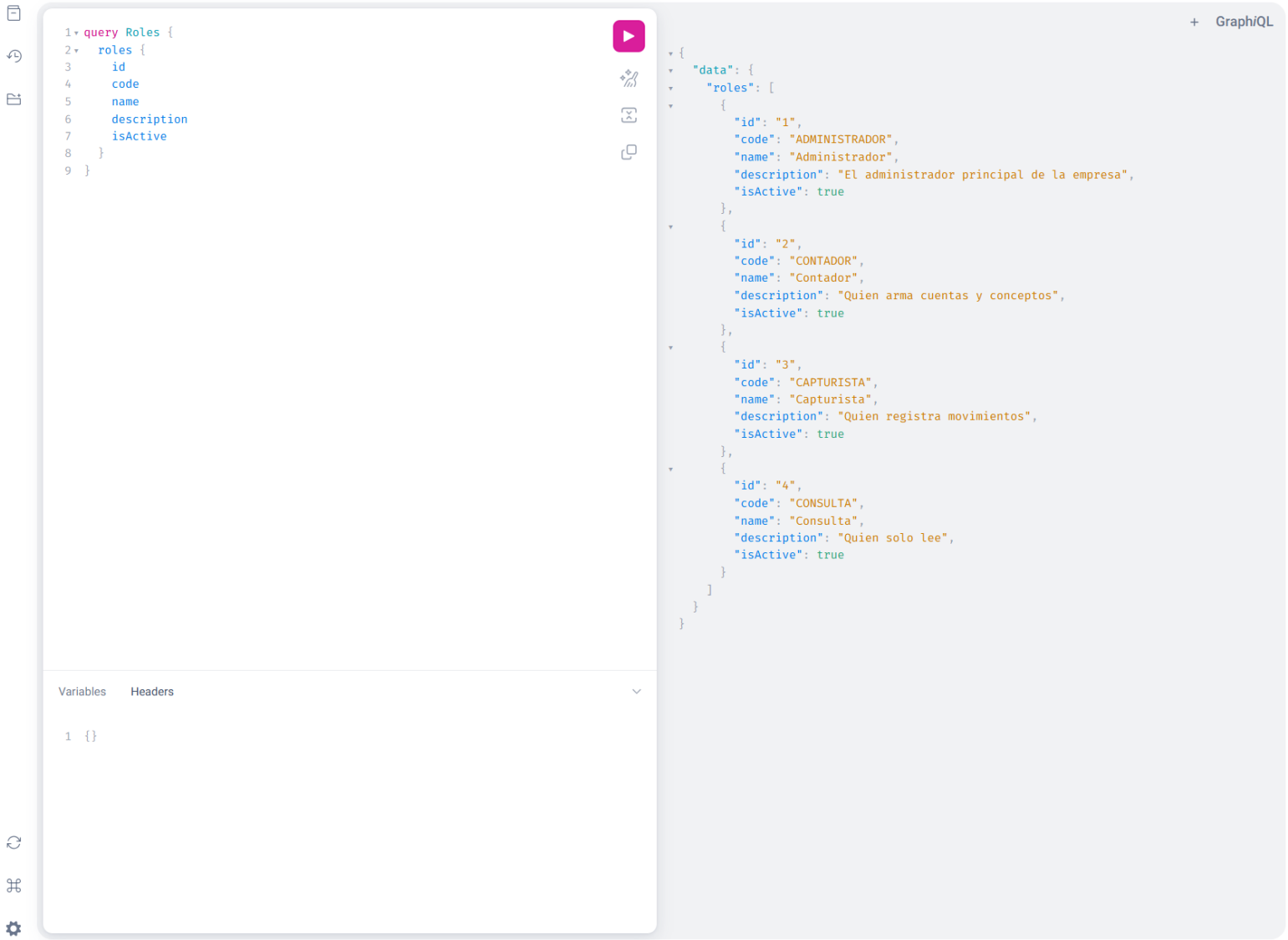
Cada captura se tomó en GraphiQL (`/graphql`) o en la consola de MySQL del contenedor. Debajo del título de cada página va la regla que demuestra.

1. Consulta roles
2. Consulta permissions
3. rolePermissions del CAPTURISTA
4. rolePermissions de CONSULTA
5. Crear empresa
6. createCompanyAdmin asigna el rol ADMINISTRADOR
7. Segundo createCompanyAdmin rechazado
8. Crear usuario contador
9. assignRole: contador recibe CONTADOR
10. Crear usuario capturista
11. assignRole: capturista recibe CAPTURISTA
12. Crear usuario consulta
13. createAccount aceptado para el CONTADOR
14. createAccount rechazado para el CAPTURISTA
15. createConcept por el CONTADOR
16. assignConceptToAccount por el CONTADOR
17. login del CAPTURISTA

18. Movimiento aceptado (CAPTURISTA)
19. login de CONSULTA
20. Movimiento rechazado (CONSULTA)
21. La tabla de movimientos no ganó la fila rechazada
22. assignRole: CAPTURISTA pasa a CONTADOR
23. Después de assignRole, ya puede crear cuentas
24. Membresías y su rol activo
25. MySQL: SELECT de roles y permissions
26. MySQL: matriz en role_permissions
27. MySQL: company_user_roles
28. Roles después de reiniciar el servidor
29. MySQL: conteos después de reiniciar

## 1. Consulta roles

**Regla que demuestra:** El catálogo de roles tiene los cuatro códigos del documento: ADMINISTRADOR, CONTADOR, CAPTURISTA y CONSULTA.



```
query Roles {
  roles {
    id
    code
    name
    description
    isActive
  }
}
```

Variables Headers

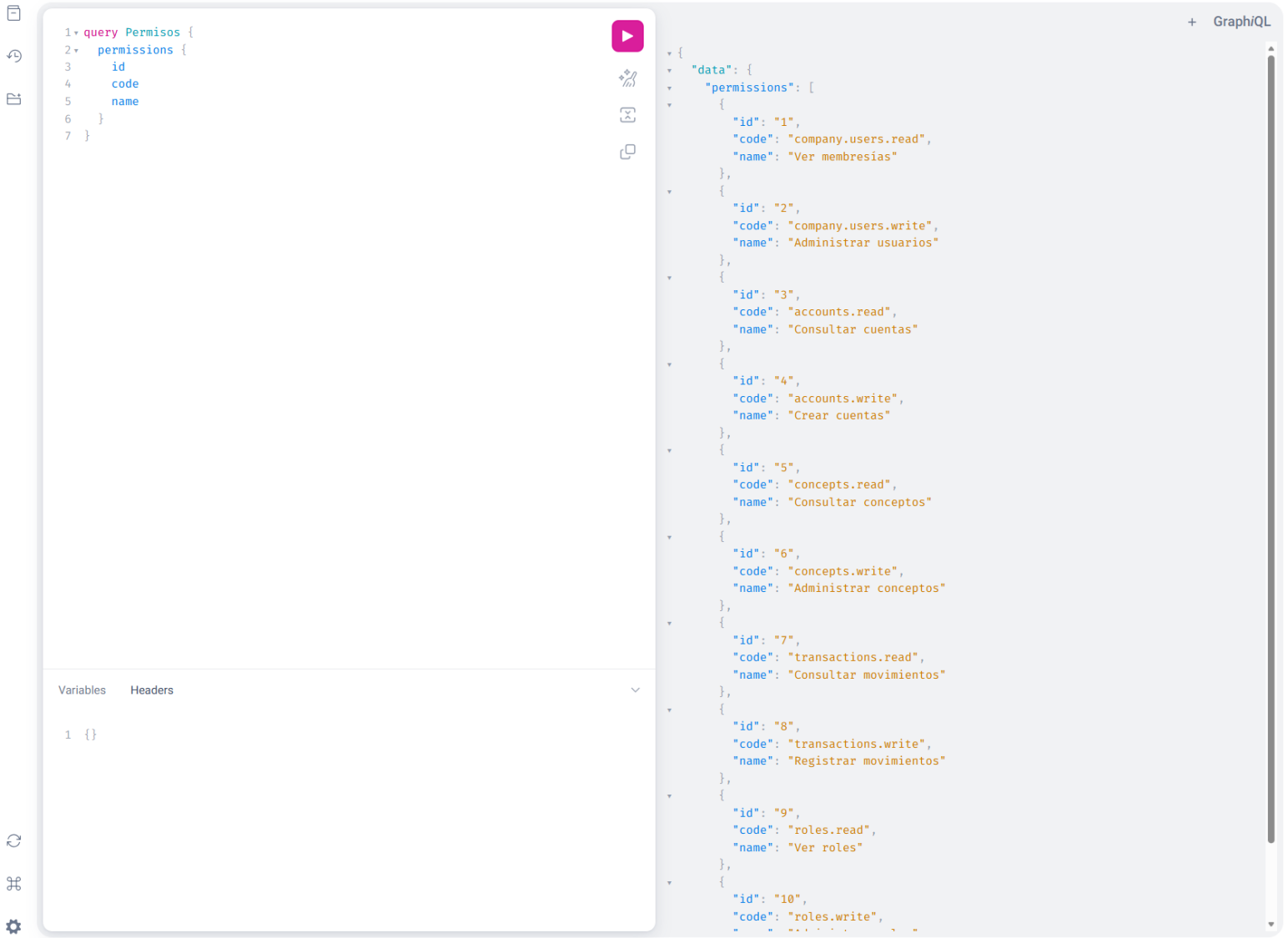
```
{}
```

GraphiQL

```
{
  "data": {
    "roles": [
      {
        "id": "1",
        "code": "ADMINISTRADOR",
        "name": "Administrador",
        "description": "El administrador principal de la empresa",
        "isActive": true
      },
      {
        "id": "2",
        "code": "CONTADOR",
        "name": "Contador",
        "description": "Quien arma cuentas y conceptos",
        "isActive": true
      },
      {
        "id": "3",
        "code": "CAPTURISTA",
        "name": "Capturista",
        "description": "Quien registra movimientos",
        "isActive": true
      },
      {
        "id": "4",
        "code": "CONSULTA",
        "name": "Consulta",
        "description": "Quien solo lee",
        "isActive": true
      }
    ]
  }
}
```

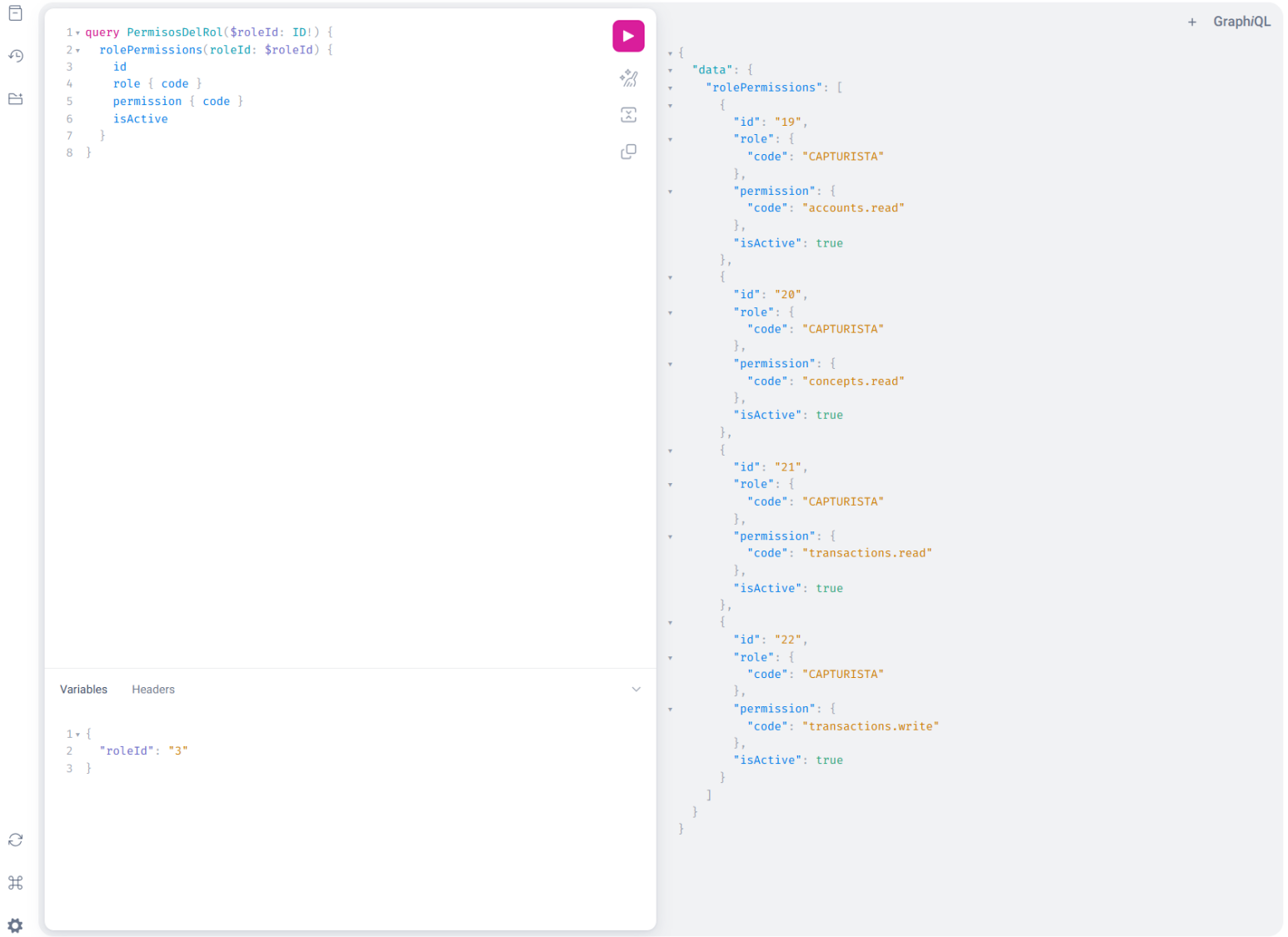
## 2. Consulta permissions

**Regla que demuestra:** El catálogo de permisos tiene los diez códigos recurso.accion del documento.



## 3. rolePermissions del CAPTURISTA

**Regla que demuestra:** El capturista tiene transactions.write y no tiene accounts.write: registra movimientos pero no crea cuentas.



```
query PermisosDelRol($roleId: ID!) {
  rolePermissions(roleId: $roleId) {
    id
    role { code }
    permission { code }
    isActive
  }
}
```

Variables

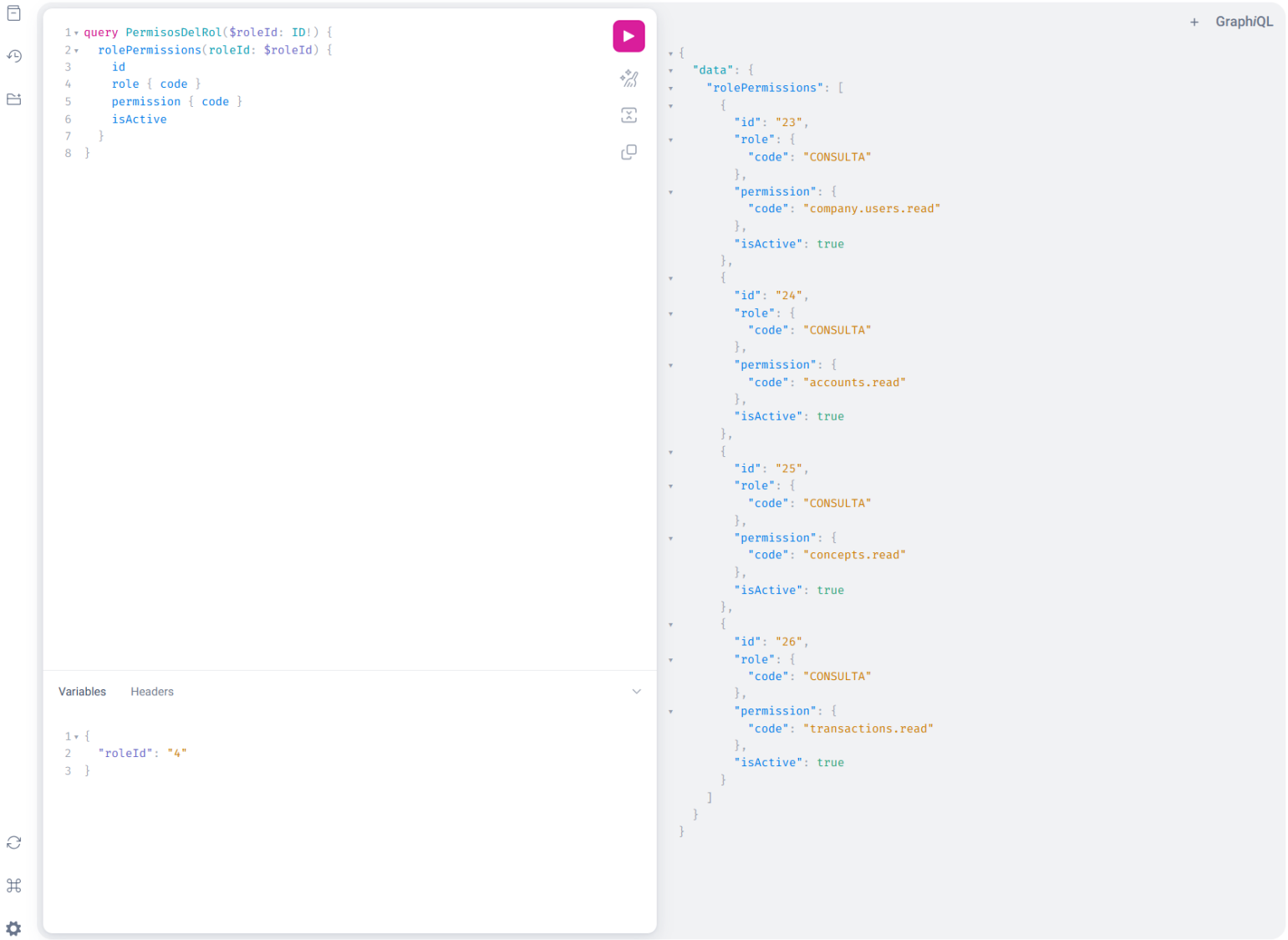
```
{
  "roleId": "3"
}
```

GraphiQL

```
{
  "data": {
    "rolePermissions": [
      {
        "id": "19",
        "role": {
          "code": "CAPTURISTA"
        },
        "permission": {
          "code": "accounts.read"
        },
        "isActive": true
      },
      {
        "id": "20",
        "role": {
          "code": "CAPTURISTA"
        },
        "permission": {
          "code": "concepts.read"
        },
        "isActive": true
      },
      {
        "id": "21",
        "role": {
          "code": "CAPTURISTA"
        },
        "permission": {
          "code": "transactions.read"
        },
        "isActive": true
      },
      {
        "id": "22",
        "role": {
          "code": "CAPTURISTA"
        },
        "permission": {
          "code": "transactions.write"
        },
        "isActive": true
      }
    ]
  }
}
```

## 4. rolePermissions de CONSULTA

**Regla que demuestra:** El rol CONSULTA no tiene ningún permiso que termine en .write: solo lee.



```
query PermisosDelRol($roleId: ID!) {
  rolePermissions(roleId: $roleId) {
    id
    role { code }
    permission { code }
    isActive
  }
}
```

Variables

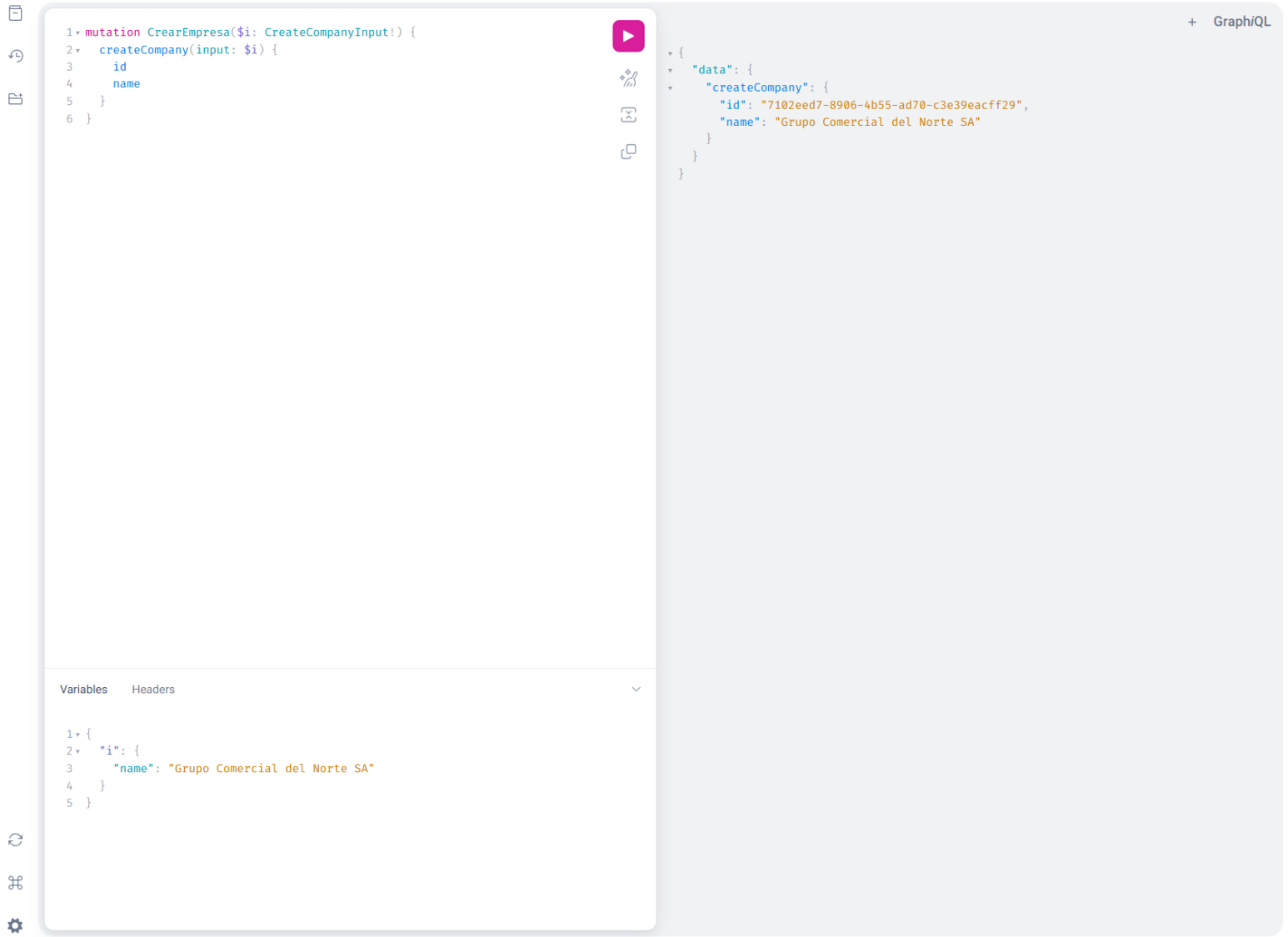
```
{
  "roleId": "4"
}
```

GraphiQL


```
{
  "data": {
    "rolePermissions": [
      {
        "id": "23",
        "role": {
          "code": "CONSULTA"
        },
        "permission": {
          "code": "company.users.read"
        },
        "isActive": true
      },
      {
        "id": "24",
        "role": {
          "code": "CONSULTA"
        },
        "permission": {
          "code": "accounts.read"
        },
        "isActive": true
      },
      {
        "id": "25",
        "role": {
          "code": "CONSULTA"
        },
        "permission": {
          "code": "concepts.read"
        },
        "isActive": true
      },
      {
        "id": "26",
        "role": {
          "code": "CONSULTA"
        },
        "permission": {
          "code": "transactions.read"
        },
        "isActive": true
      }
    ]
  }
}
```

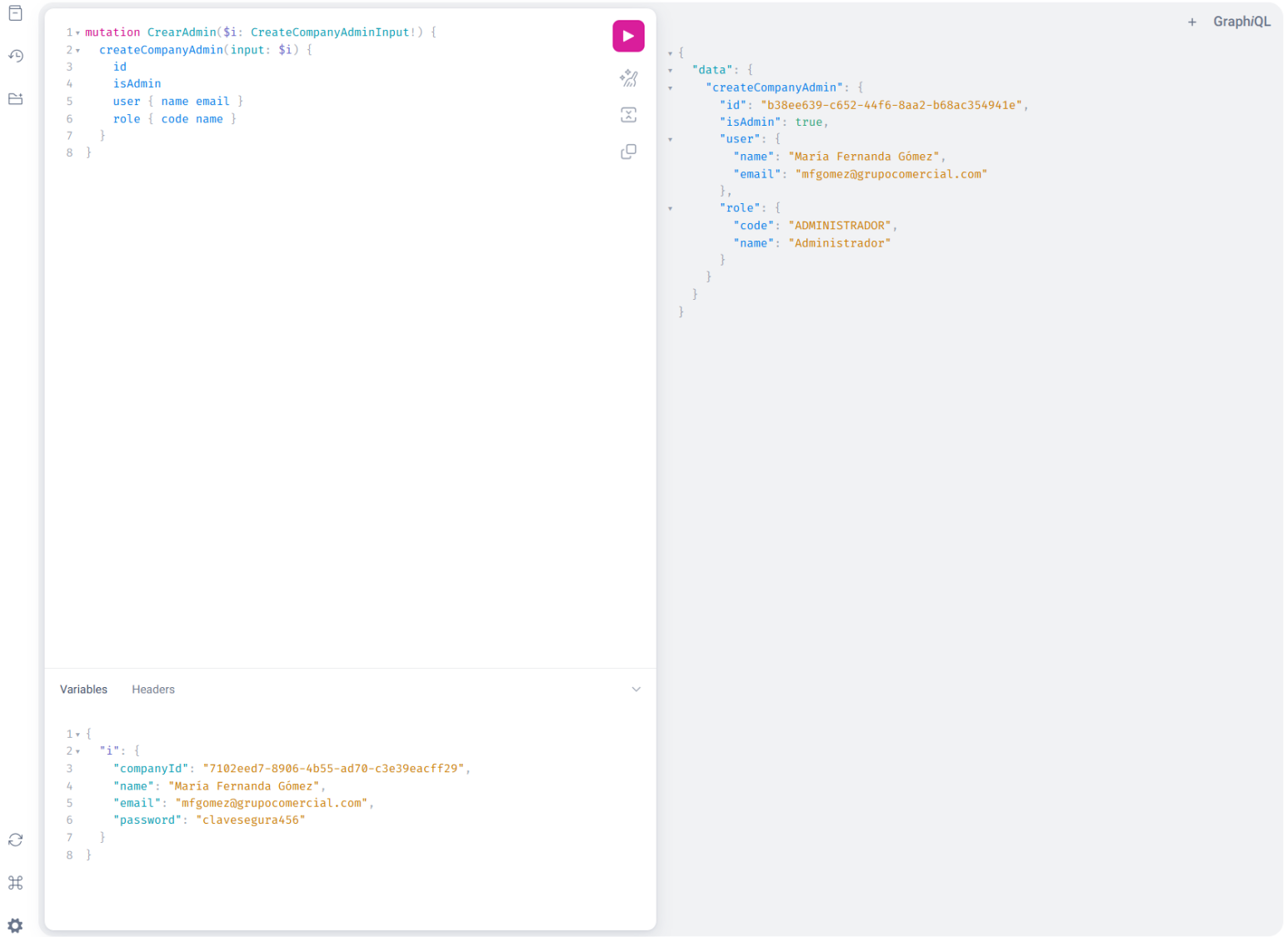
## 5. Crear empresa

**Regla que demuestra:** La empresa es la raíz: todo lo demás (membresías, cuentas, movimientos) cuelga de ella.



## 6. createCompanyAdmin asigna el rol ADMINISTRADOR

**Regla que demuestra:** El administrador principal conserva isAdmin = true y además recibe el rol ADMINISTRADOR.



```
mutation CrearAdmin($i: CreateCompanyAdminInput!) {
  createCompanyAdmin(input: $i) {
    id
    isAdmin
    user { name email }
    role { code name }
  }
}
```

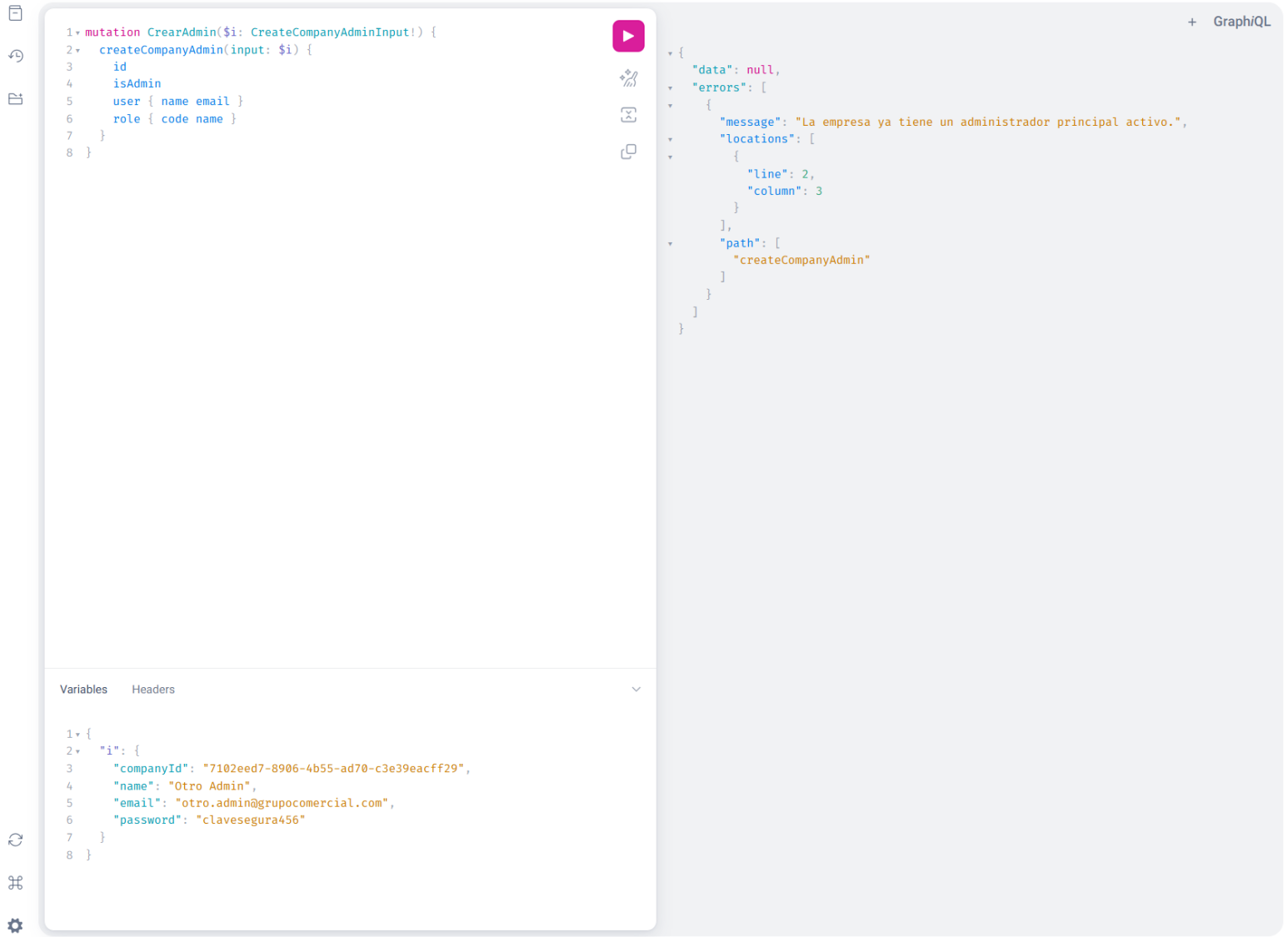
```
{
  "data": {
    "createCompanyAdmin": {
      "id": "b38ee639-c652-44f6-8aa2-b68ac354941e",
      "isAdmin": true,
      "user": {
        "name": "María Fernanda Gómez",
        "email": "mfgomez@grupocomercial.com"
      },
      "role": {
        "code": "ADMINISTRADOR",
        "name": "Administrador"
      }
    }
  }
}
```

Variables Headers

```
{
  "i": {
    "companyId": "7102eed7-8906-4b55-ad70-c3e39eacff29",
    "name": "María Fernanda Gómez",
    "email": "mfgomez@grupocomercial.com",
    "password": "clavesegura456"
  }
}
```

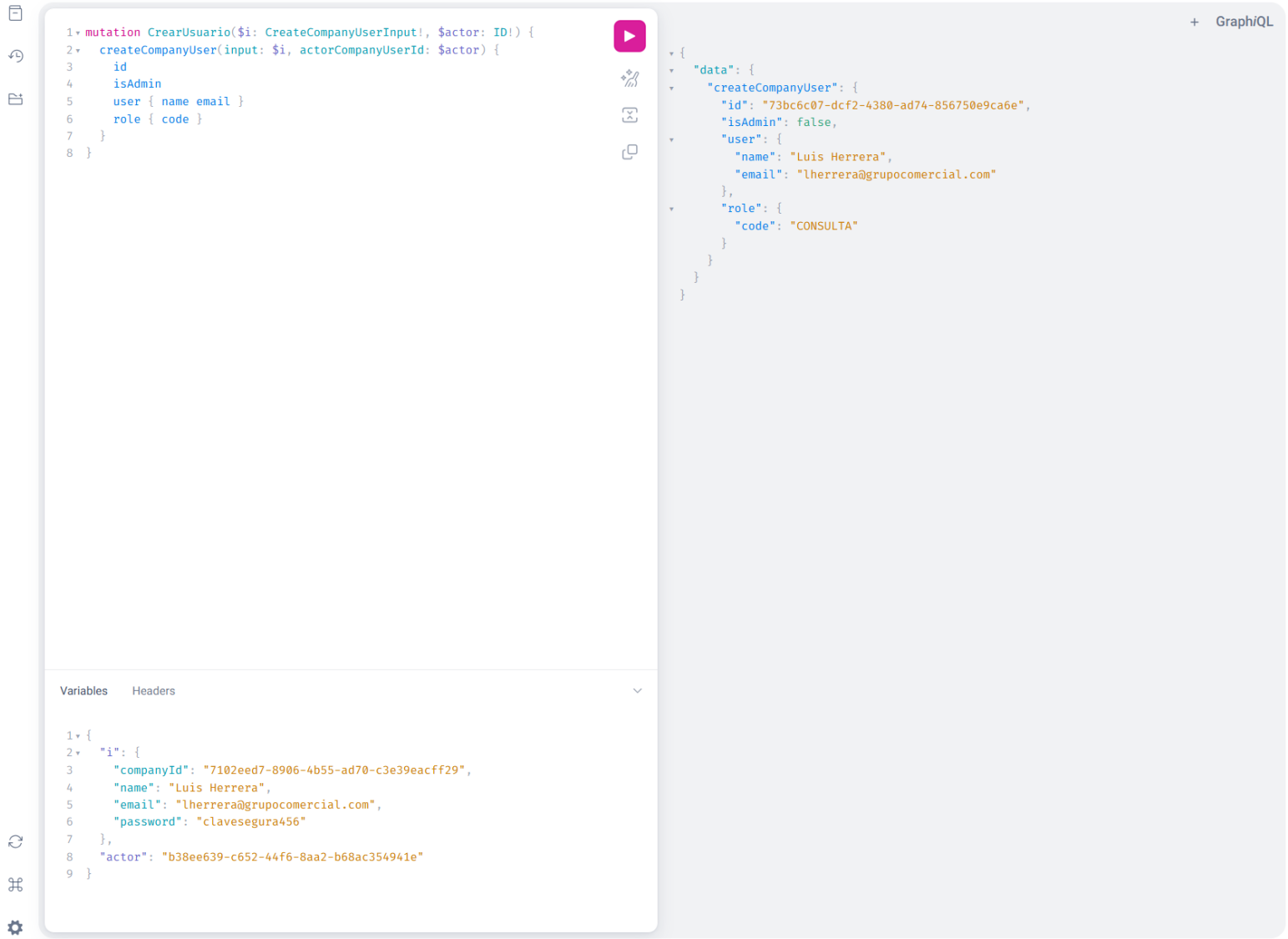
## 7. Segundo createCompanyAdmin rechazado

**Regla que demuestra:** La regla de un solo administrador principal por empresa sigue intacta.



## 8. Crear usuario contador

**Regla que demuestra:** createCompanyUser exige company.users.write al actor (el administrador) y la membresía nace con el rol CONSULTA.



```
mutation CrearUsuario($i: CreateCompanyUserInput!, $actor: ID!) {
  createCompanyUser(input: $i, actorCompanyUserId: $actor) {
    id
    isAdmin
    user { name email }
    role { code }
  }
}
```

GraphiQL

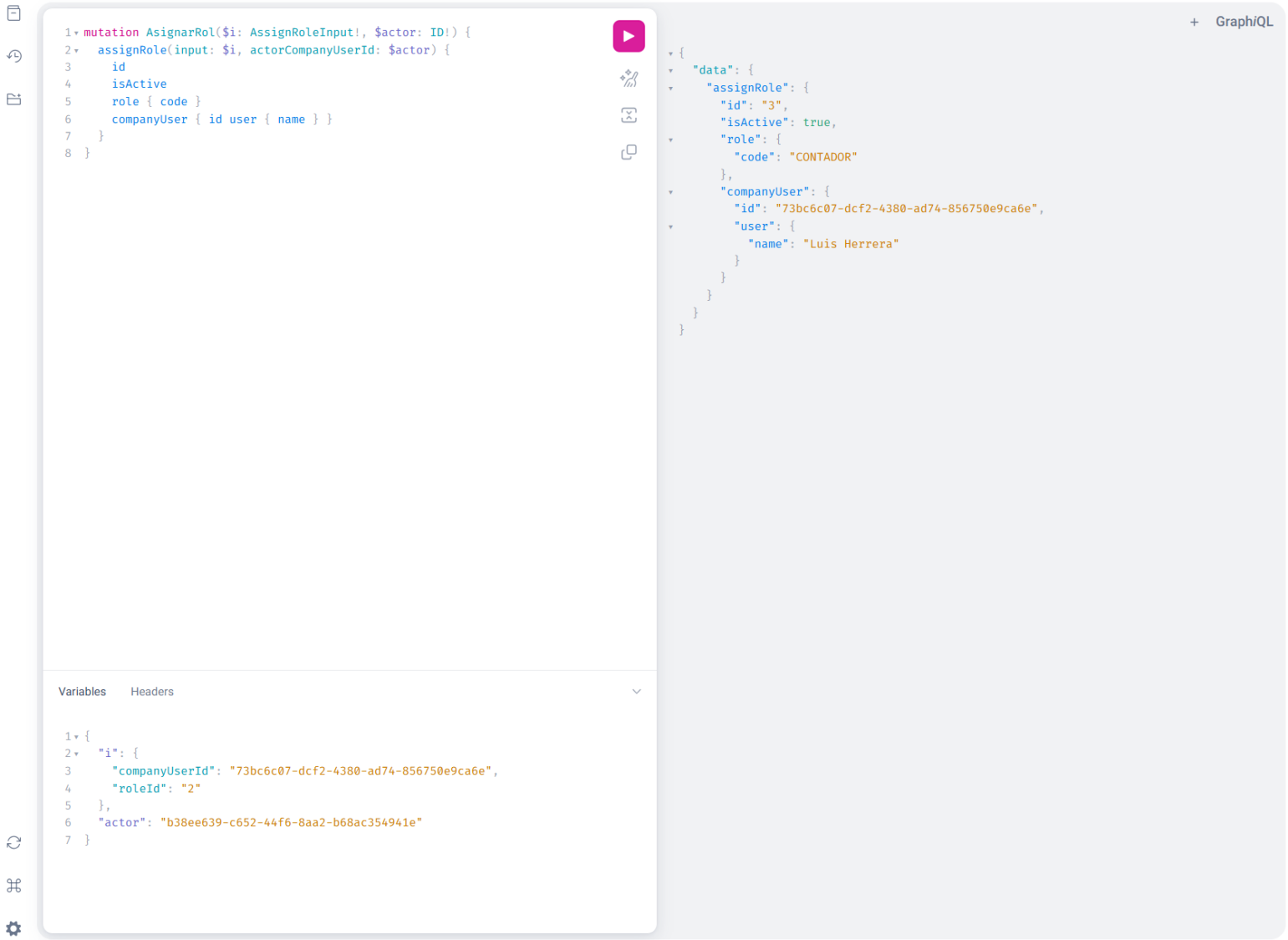
```
{
  "data": {
    "createCompanyUser": {
      "id": "73bc6c07-dcf2-4380-ad74-856750e9ca6e",
      "isAdmin": false,
      "user": {
        "name": "Luis Herrera",
        "email": "lherrera@grupocomercial.com"
      },
      "role": {
        "code": "CONSULTA"
      }
    }
  }
}
```

Variables Headers

```
{
  "i": {
    "companyId": "7102eed7-8906-4b55-ad70-c3e39eacff29",
    "name": "Luis Herrera",
    "email": "lherrera@grupocomercial.com",
    "password": "clavesegura456"
  },
  "actor": "b38ee639-c652-44f6-8aa2-b68ac354941e"
}
```

## 9. assignRole: contador recibe CONTADOR

**Regla que demuestra:** El rol vive en la membresía (company_user_roles), no en User. El administrador tiene roles.write y asigna CONTADOR.



```
mutation AsignarRol($i: AssignRoleInput!, $actor: ID!) {
  assignRole(input: $i, actorCompanyUserId: $actor) {
    id
    isActive
    role { code }
    companyUser { id user { name } }
  }
}
```

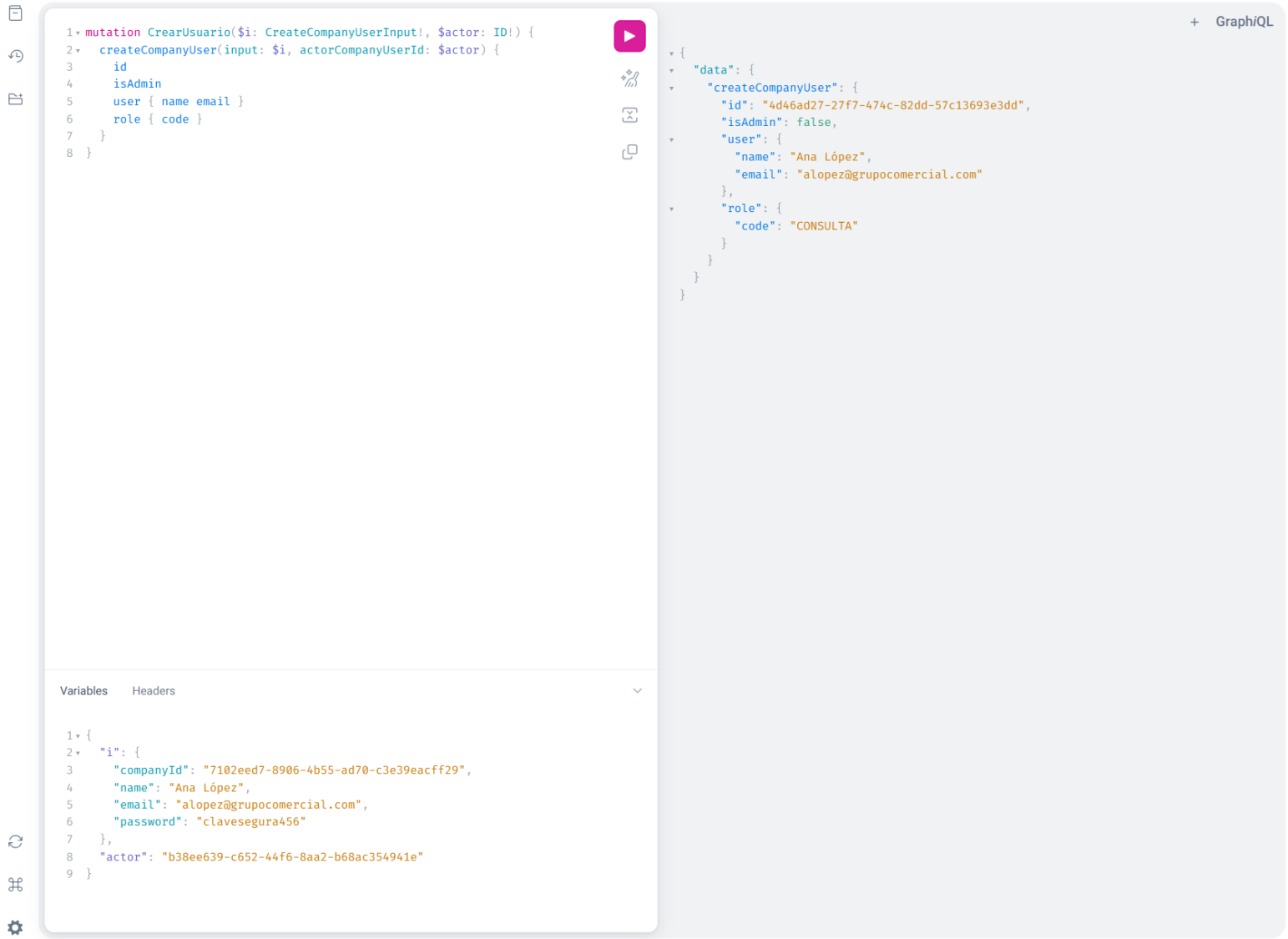
```
{
  "data": {
    "assignRole": {
      "id": "3",
      "isActive": true,
      "role": {
        "code": "CONTADOR"
      },
      "companyUser": {
        "id": "73bc6c07-dcf2-4380-ad74-856750e9ca6e",
        "user": {
          "name": "Luis Herrera"
        }
      }
    }
  }
}
```

Variables Headers

```
{
  "i": {
    "companyUserId": "73bc6c07-dcf2-4380-ad74-856750e9ca6e",
    "roleId": "2"
  },
  "actor": "b38ee639-c652-44f6-8aa2-b68ac354941e"
}
```

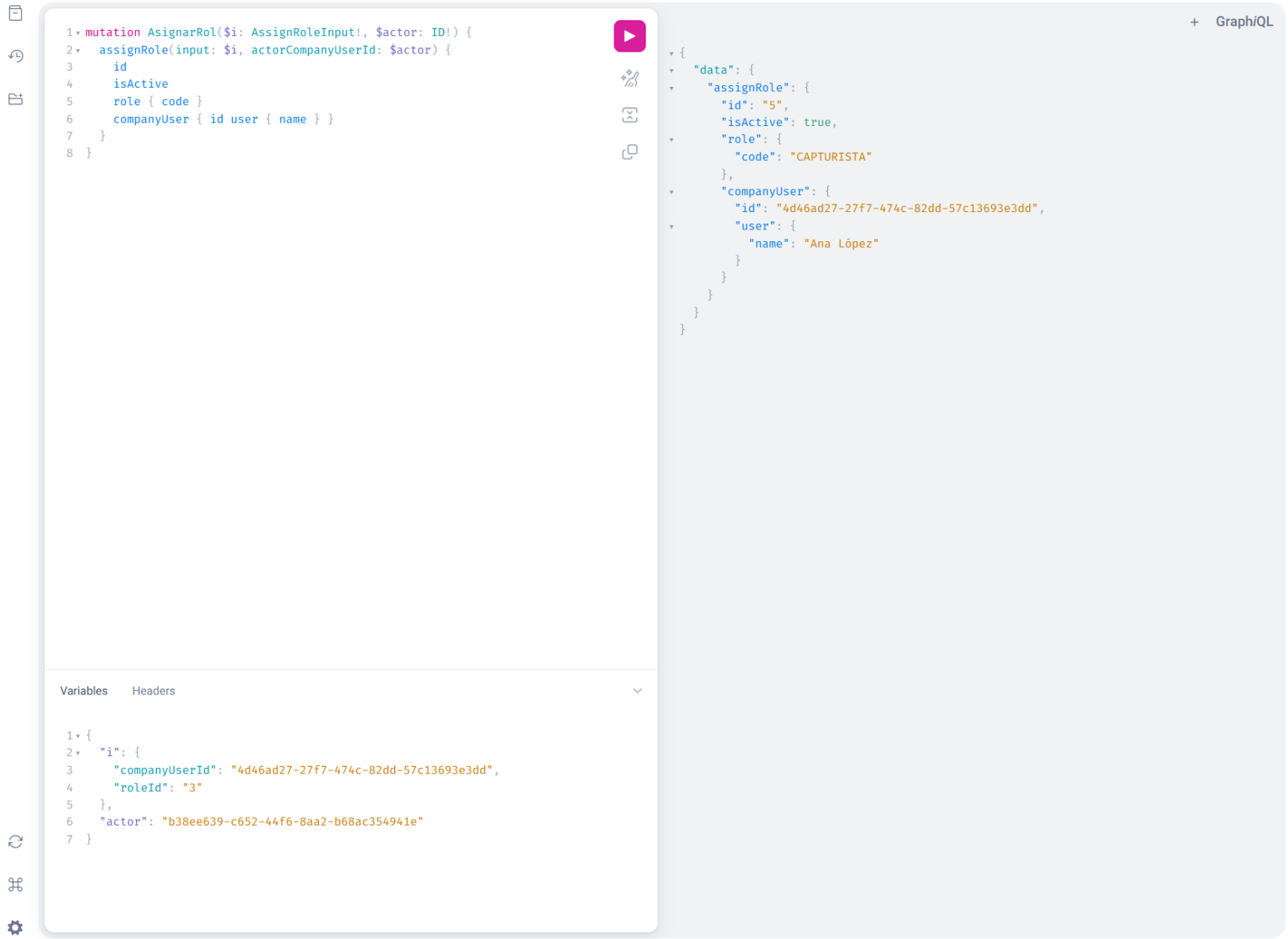
## 10. Crear usuario capturista

**Regla que demuestra:** createCompanyUser exige company.users.write al actor (el administrador) y la membresía nace con el rol CONSULTA.



## 11. assignRole: capturista recibe CAPTURISTA

**Regla que demuestra:** El rol vive en la membresía (company_user_roles), no en User. El administrador tiene roles.write y asigna CAPTURISTA.



```
mutation AsignarRol($i: AssignRoleInput!, $actor: ID!) {
  assignRole(input: $i, actorCompanyUserId: $actor) {
    id
    isActive
    role { code }
    companyUser { id user { name } }
  }
}
```

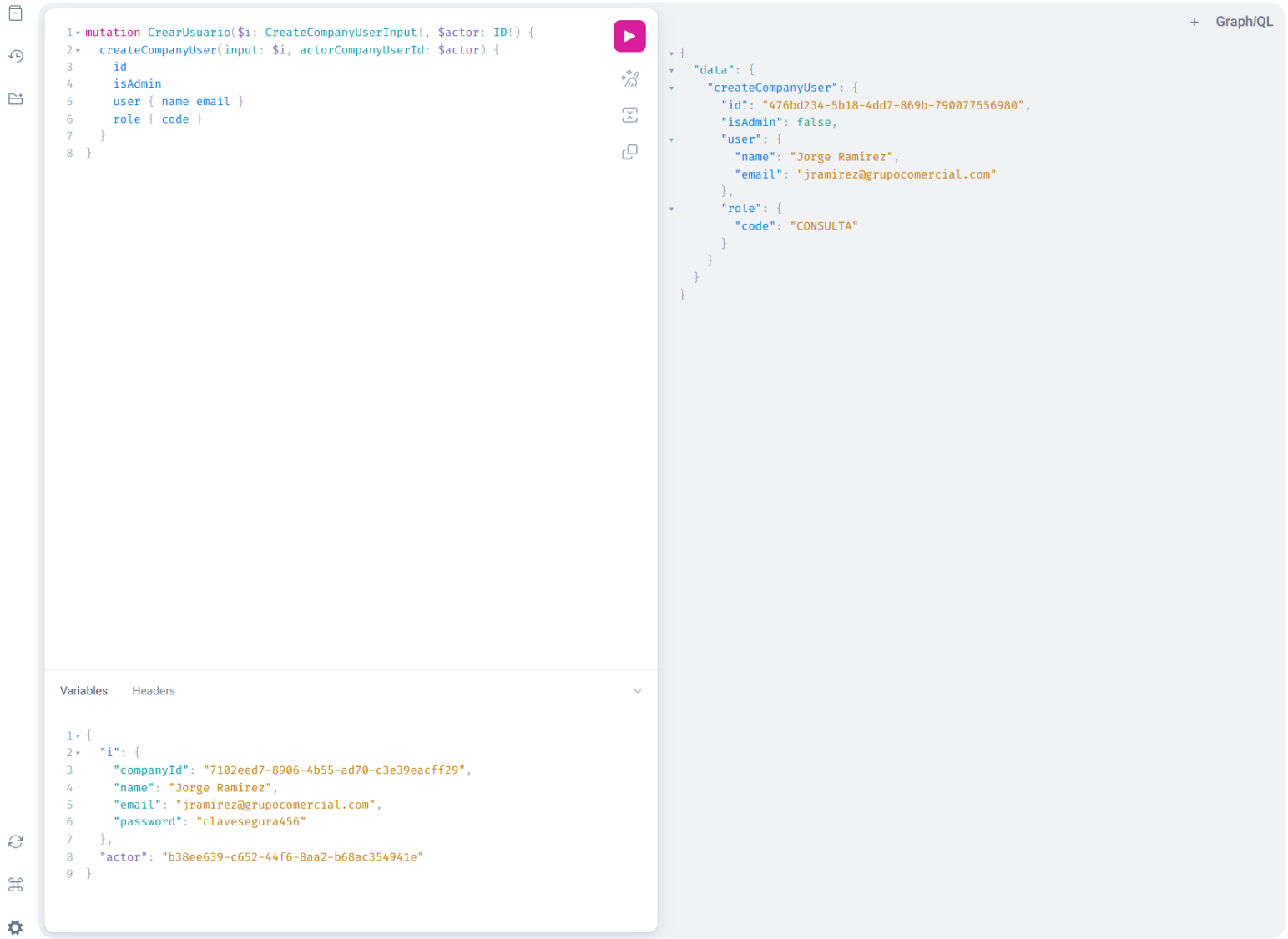
```
{
  "data": {
    "assignRole": {
      "id": "5",
      "isActive": true,
      "role": {
        "code": "CAPTURISTA"
      },
      "companyUser": {
        "id": "4d46ad27-27f7-474c-82dd-57c13693e3dd",
        "user": {
          "name": "Ana López"
        }
      }
    }
  }
}
```

Variables Headers

```
{
  "i": {
    "companyUserId": "4d46ad27-27f7-474c-82dd-57c13693e3dd",
    "roleId": "3"
  },
  "actor": "b38ee639-c652-44f6-8aa2-b68ac354941e"
}
```

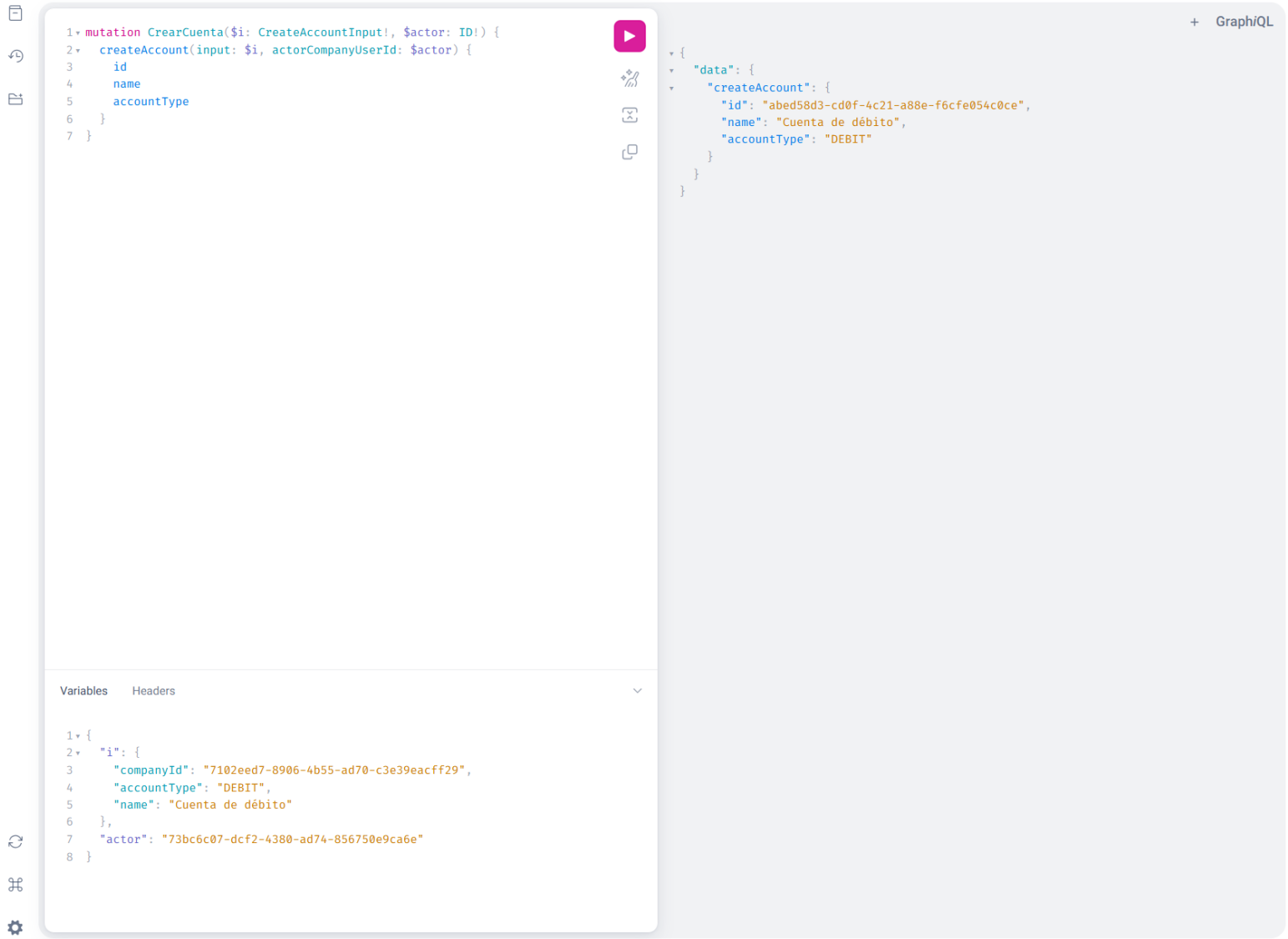
## 12. Crear usuario consulta

**Regla que demuestra:** createCompanyUser exige company.users.write al actor (el administrador) y la membresía nace con el rol CONSULTA.



## 13. createAccount aceptado para el CONTADOR

**Regla que demuestra:** El contador tiene accounts.write, así que la cuenta se guarda.



```
mutation CrearCuenta($i: CreateAccountInput!, $actor: ID!) {
  createAccount(input: $i, actorCompanyUserId: $actor) {
    id
    name
    accountType
  }
}
```

GraphiQL

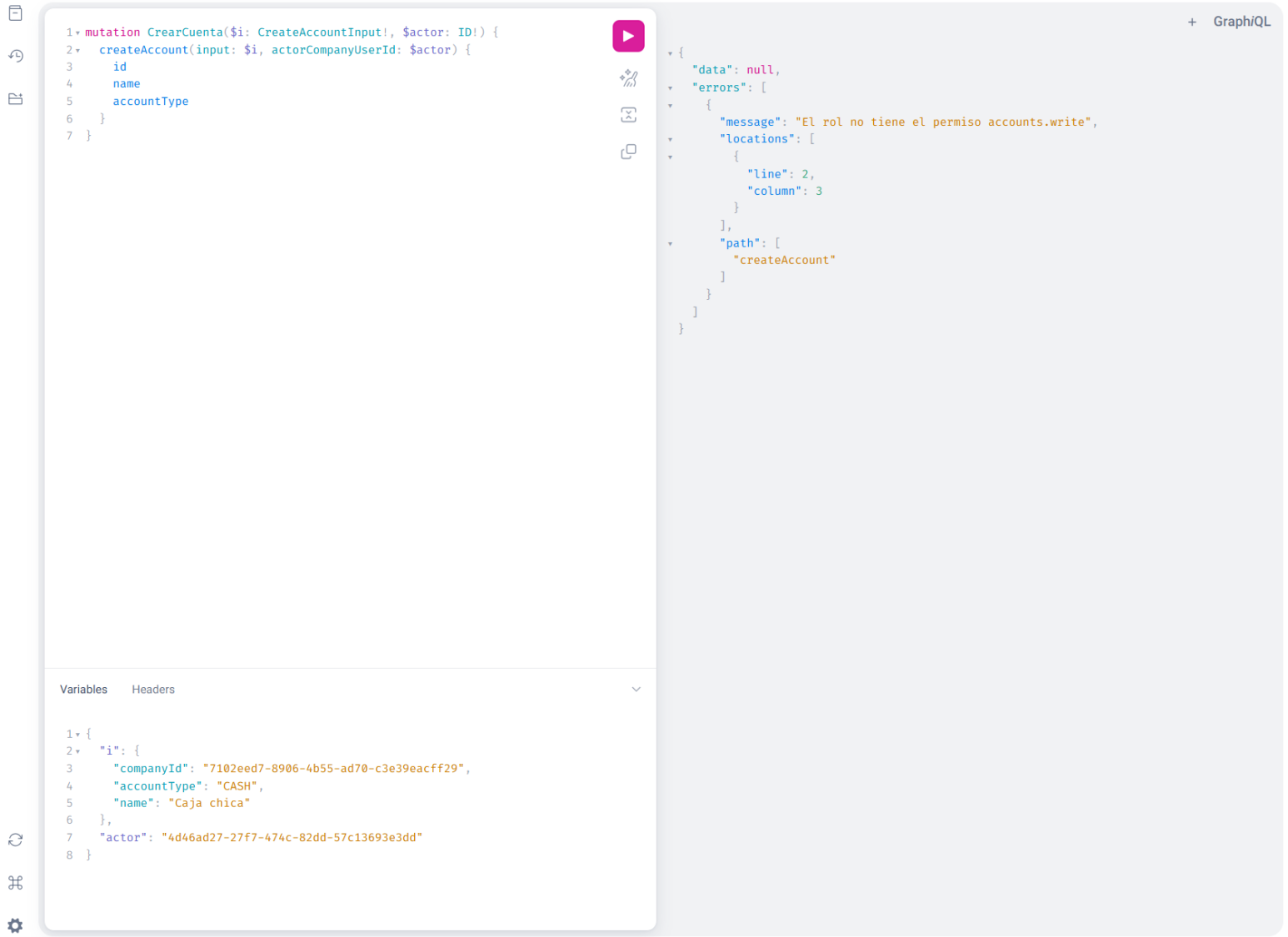
```
{
  "data": {
    "createAccount": {
      "id": "abed58d3-cd0f-4c21-a88e-f6cfe054c0ce",
      "name": "Cuenta de débito",
      "accountType": "DEBIT"
    }
  }
}
```

Variables Headers

```
{
  "i": {
    "companyId": "7102eed7-8906-4b55-ad70-c3e39eacff29",
    "accountType": "DEBIT",
    "name": "Cuenta de débito"
  },
  "actor": "73bc6c07-dcf2-4380-ad74-856750e9ca6e"
}
```

## 14. createAccount rechazado para el CAPTURISTA

**Regla que demuestra:** El capturista no tiene accounts.write: el resolver lanza el error antes del commit y la cuenta no se crea.



```
mutation CrearCuenta($i: CreateAccountInput!, $actor: ID!) {
  createAccount(input: $i, actorCompanyUserId: $actor) {
    id
    name
    accountType
  }
}
```

Variables

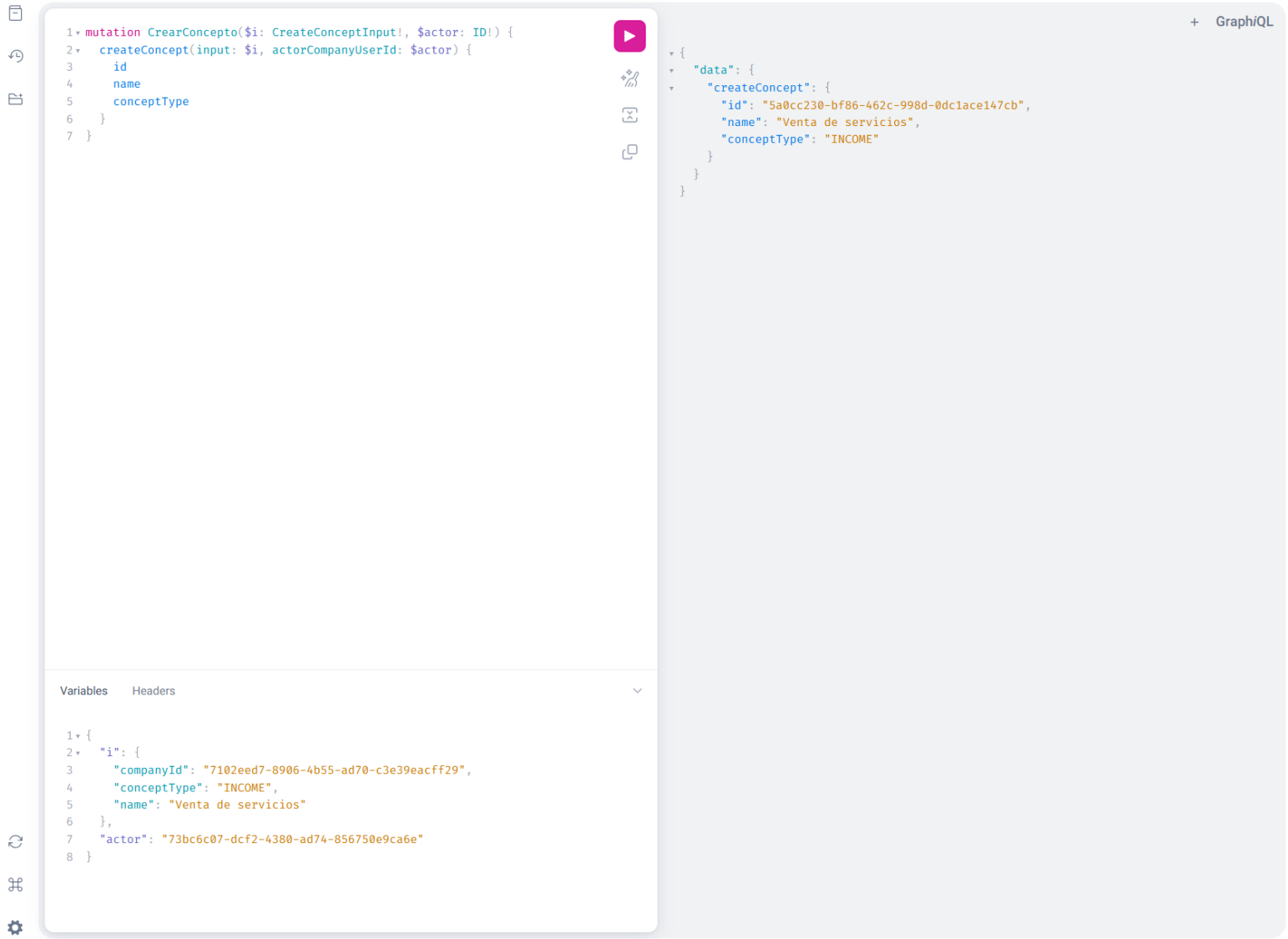
```
{
  "i": {
    "companyId": "7102eed7-8906-4b55-ad70-c3e39eacff29",
    "accountType": "CASH",
    "name": "Caja chica"
  },
  "actor": "4d46ad27-27f7-474c-82dd-57c13693e3dd"
}
```

GraphiQL

```
{
  "data": null,
  "errors": [
    {
      "message": "El rol no tiene el permiso accounts.write",
      "locations": [
        {
          "line": 2,
          "column": 3
        }
      ],
      "path": [
        "createAccount"
      ]
    }
  ]
}
```

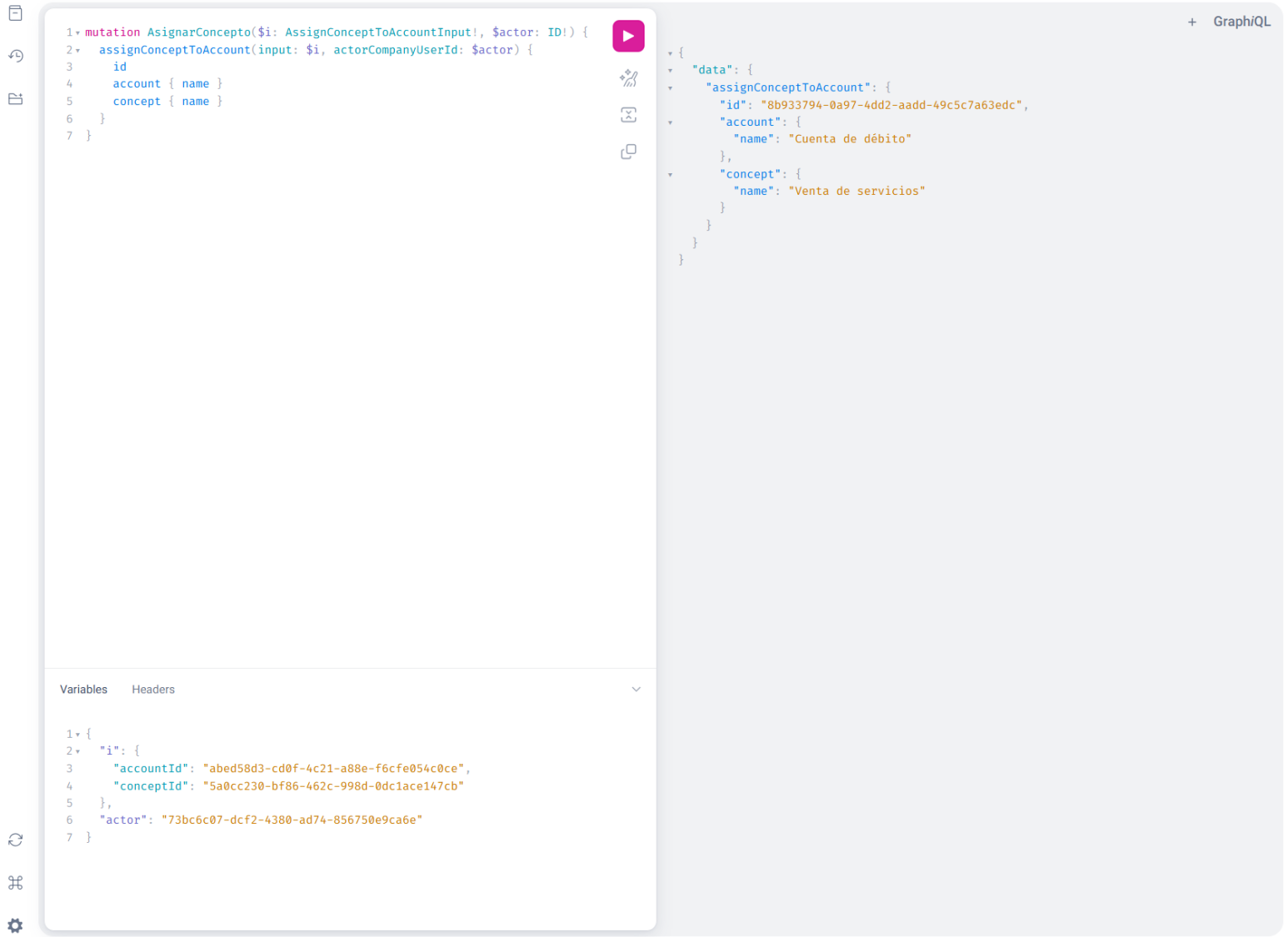
## 15. createConcept por el CONTADOR

**Regla que demuestra:** El contador tiene concepts.write y define el catálogo de conceptos.



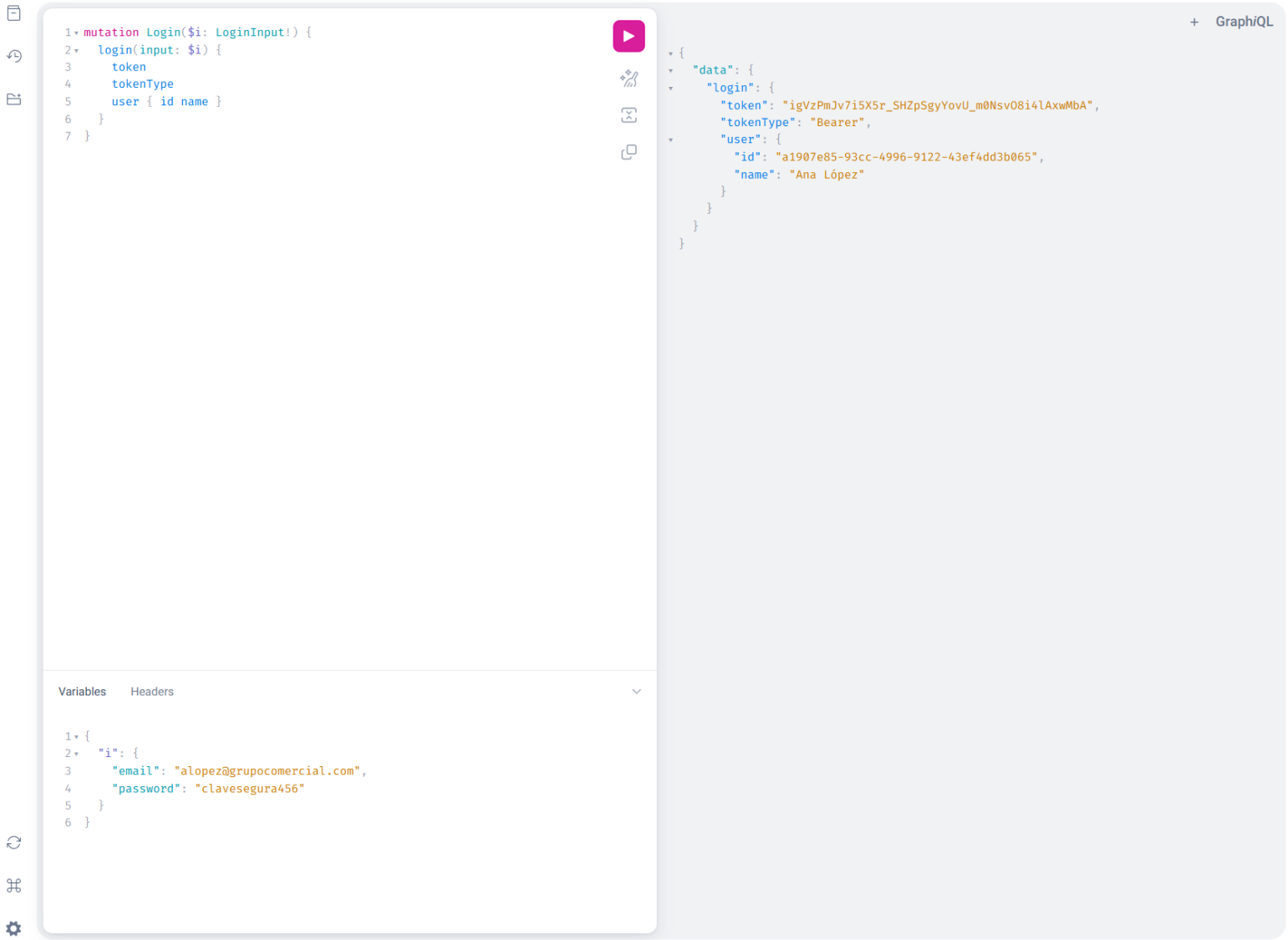
## 16. assignConceptToAccount por el CONTADOR

**Regla que demuestra:** assignConceptToAccount también exige concepts.write.



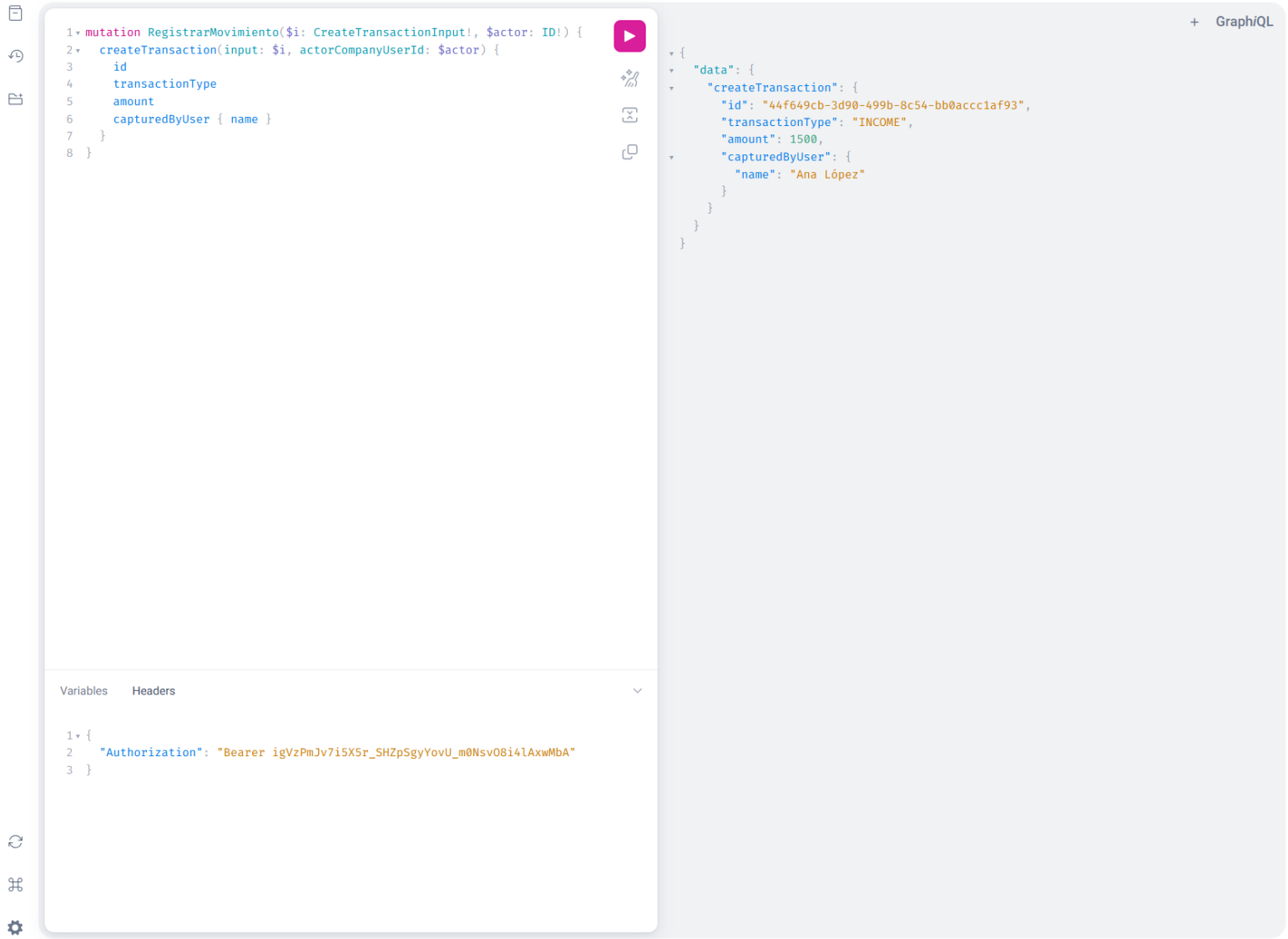
## 17. login del CAPTURISTA

**Regla que demuestra:** login no cambia: devuelve el token que createTransaction sigue pidiendo.



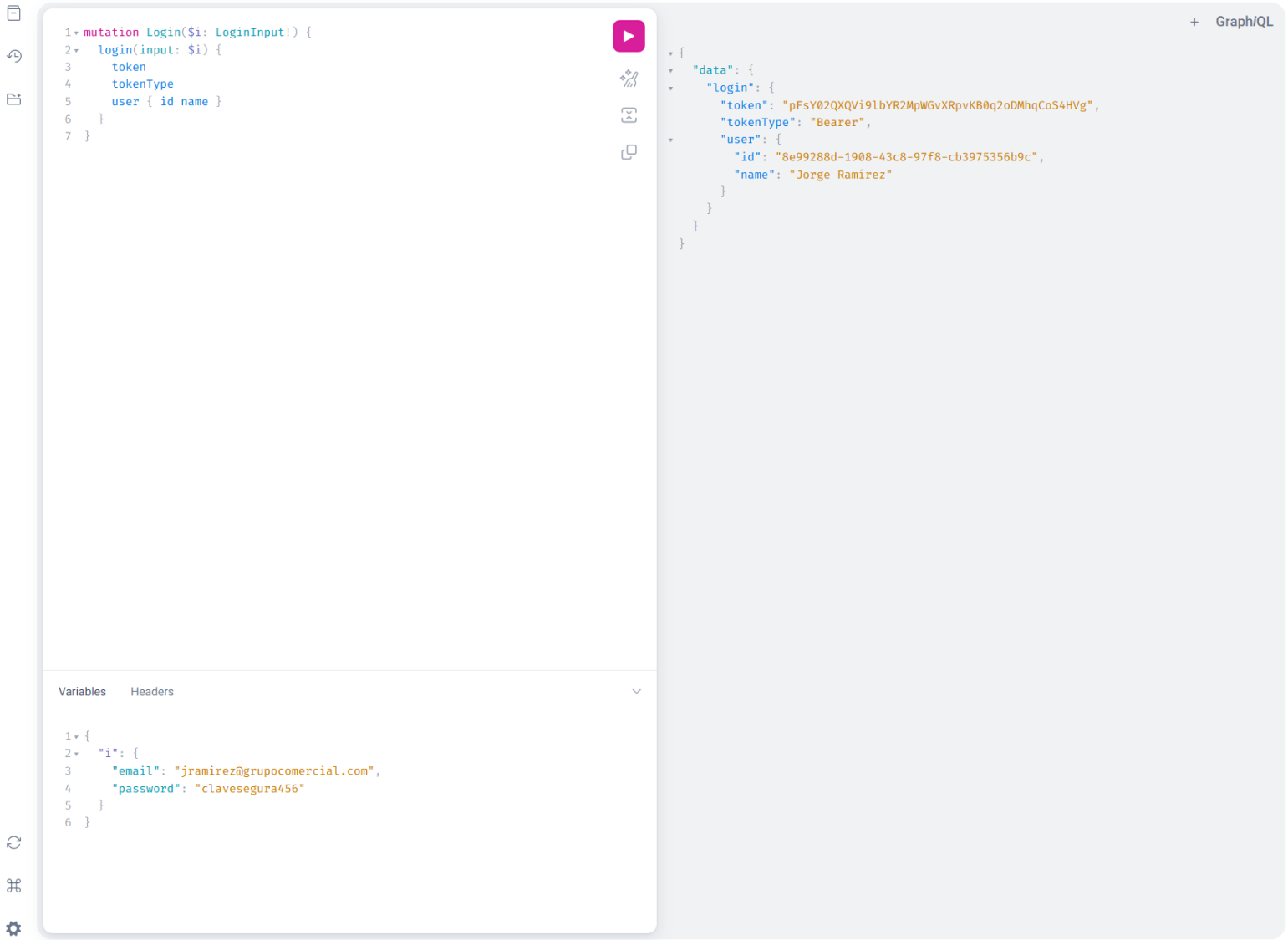
## 18. Movimiento aceptado (CAPTURISTA)

**Regla que demuestra:** El capturista tiene transactions.write: el movimiento se guarda.



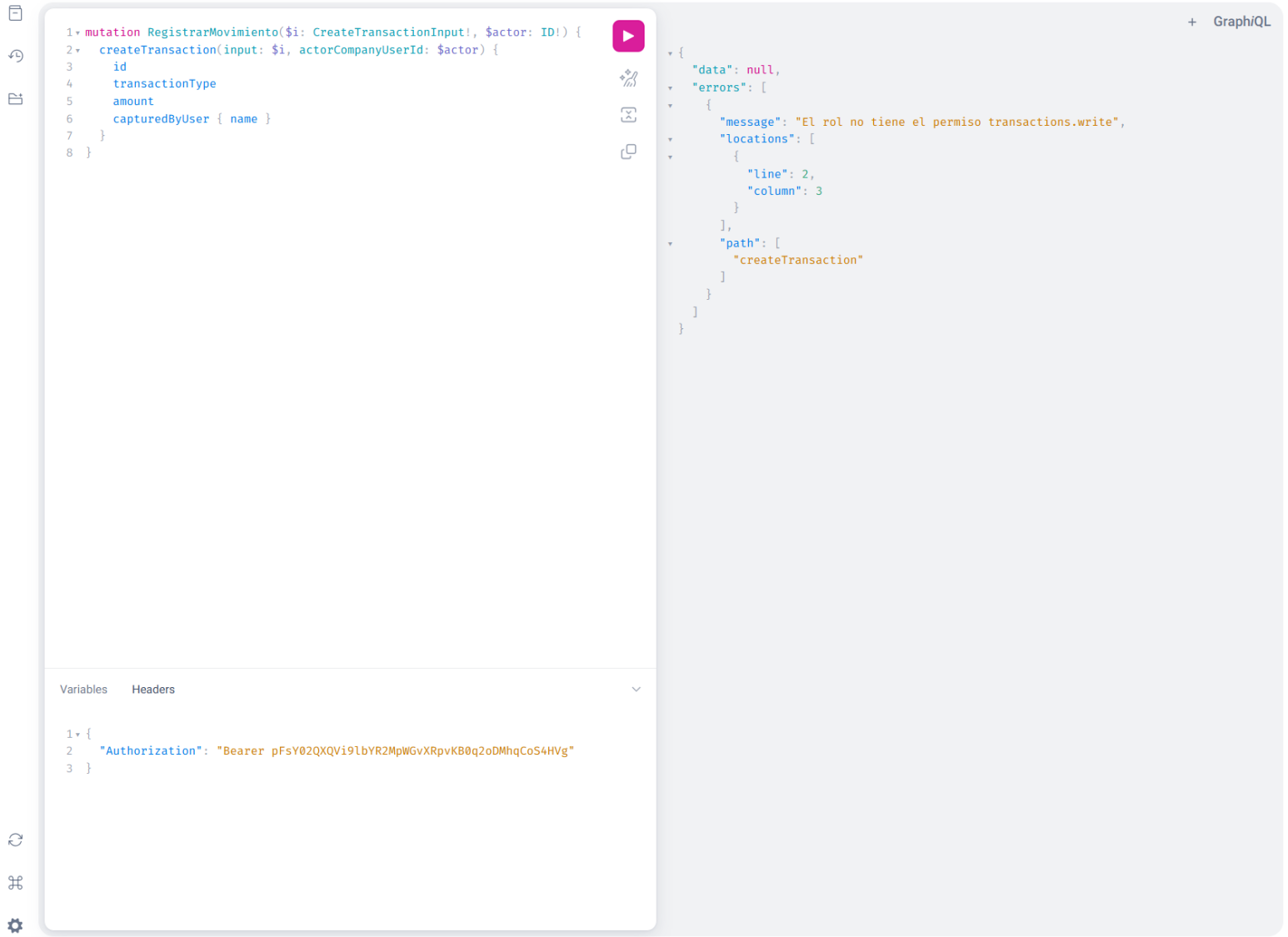
## 19. login de CONSULTA

**Regla que demuestra:** El usuario de consulta se autentica igual; su rol decide lo que puede escribir.



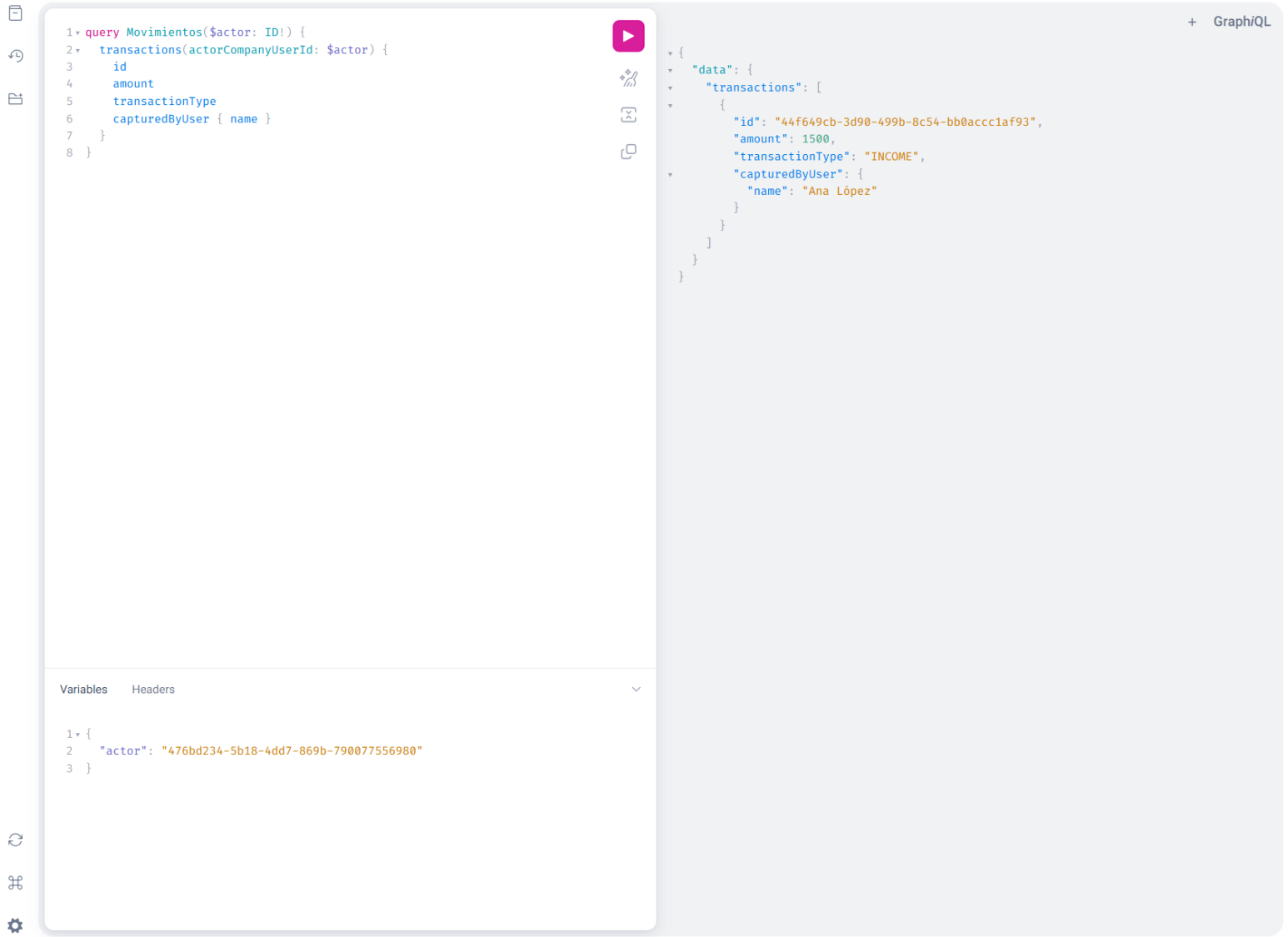
## 20. Movimiento rechazado (CONSULTA)

**Regla que demuestra:** CONSULTA no tiene transactions.write: GraphQL responde con errors y no se hace commit.



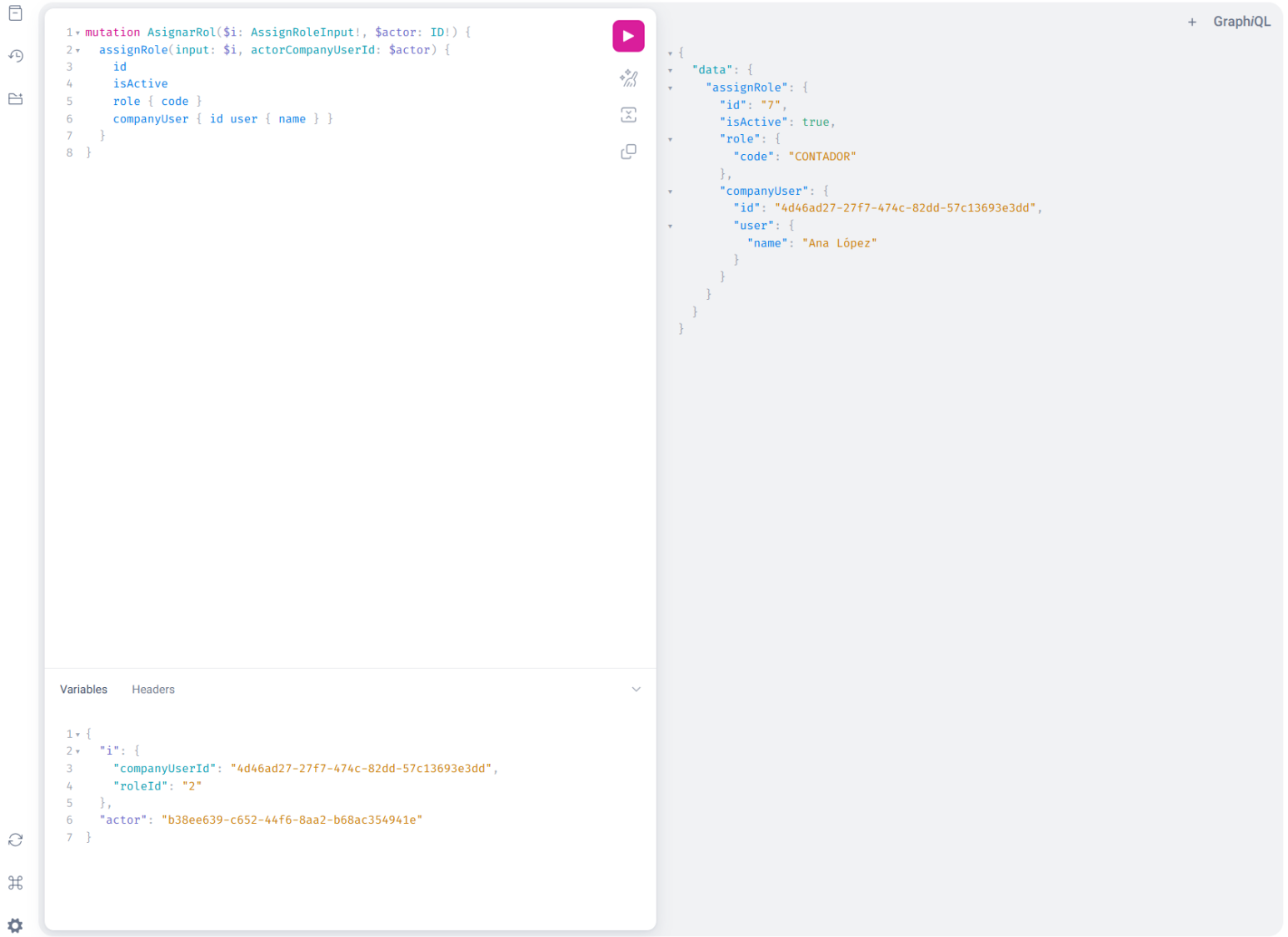
## 21. La tabla de movimientos no ganó la fila rechazada

**Regla que demuestra:** CONSULTA tiene transactions.read: lee los movimientos y solo aparece el de 1500, el de 999 no se guardó.



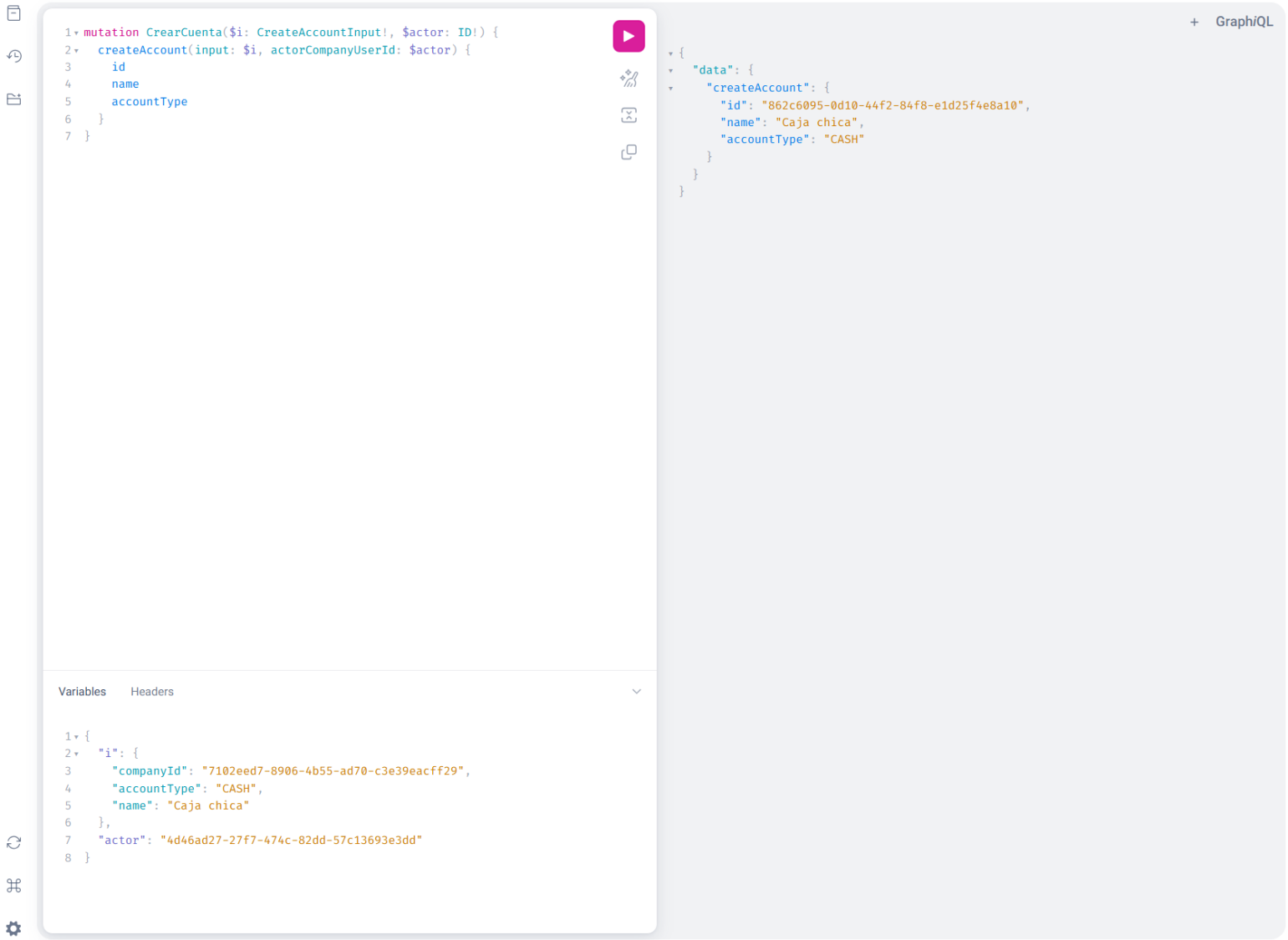
## 22. assignRole: CAPTURISTA pasa a CONTADOR

**Regla que demuestra:** assignRole desactiva el rol anterior y deja uno solo activo por membresía.



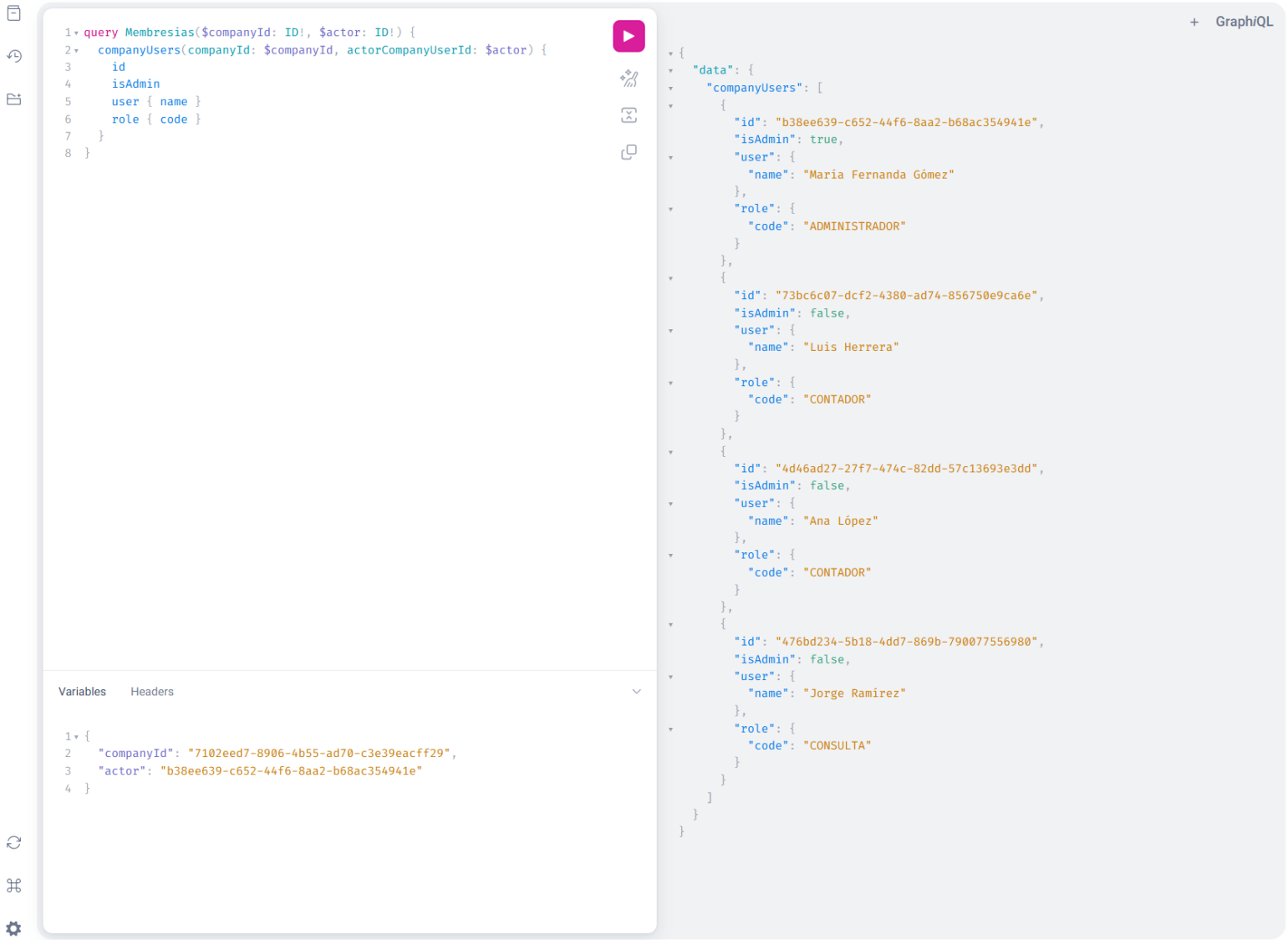
## 23. Después de assignRole, ya puede crear cuentas

**Regla que demuestra:** Con el rol CONTADOR, la misma membresía que antes fue rechazada ahora crea la cuenta.



## 24. Membresías y su rol activo

**Regla que demuestra:** Cada membresía tiene un solo rol activo; el administrador conserva isAdmin y el rol ADMINISTRADOR.



## 25. MySQL: SELECT de roles y permissions

**Regla que demuestra:** Los roles y permisos son catálogos en sus propias tablas, no columnas de User.

```
PS C:\Users\Gema\Desktop\Actividad-5> docker exec -it ingresos-mysql mysql -uingresos -p ingresos
mysql> SELECT * FROM roles;
mysql> SELECT id, code, name, is_active FROM permissions;

+----+---------------+---------------+----------------------------------------+-----------+
| id | code          | name          | description                            | is_active |
+----+---------------+---------------+----------------------------------------+-----------+
|  1 | ADMINISTRADOR | Administrador | El administrador principal de la empresa |         1 |
|  2 | CONTADOR      | Contador      | Quien arma cuentas y conceptos         |         1 |
|  3 | CAPTURISTA    | Capturista    | Quien registra movimientos             |         1 |
|  4 | CONSULTA      | Consulta      | Quien solo lee                         |         1 |
+----+---------------+---------------+----------------------------------------+-----------+
+----+--------------------+----------------------+-----------+
| id | code               | name                 | is_active |
+----+--------------------+----------------------+-----------+
|  1 | company.users.read | Ver membresías       |         1 |
|  2 | company.users.write | Administrar usuarios |         1 |
|  3 | accounts.read      | Consultar cuentas    |         1 |
|  4 | accounts.write     | Crear cuentas        |         1 |
|  5 | concepts.read      | Consultar conceptos  |         1 |
|  6 | concepts.write     | Administrar conceptos |         1 |
|  7 | transactions.read  | Consultar movimientos |         1 |
|  8 | transactions.write | Registrar movimientos |         1 |
|  9 | roles.read         | Ver roles            |         1 |
| 10 | roles.write        | Administrar roles    |         1 |
+----+--------------------+----------------------+-----------+
```

## 26. MySQL: matriz en role_permissions

**Regla que demuestra:** La matriz que vive en role_permissions coincide con la del documento (26 parejas activas).

```
PS C:\Users\Gema\Desktop\Actividad-5> docker exec -it ingresos-mysql mysql -uingresos -p ingresos
mysql> SELECT p.code AS permiso, MAX(IF(r.code='ADMINISTRADOR','sí','no')) AS ADMINISTRADOR, MAX(IF(r.code='CONTADOR','sí','no')) AS CONTADOR, MAX(IF(r.code='CAPTURISTA','sí','no')) AS CAPTURISTA,
MAX(IF(r.code='CONSULTA','sí','no')) AS CONSULTA FROM permissions p LEFT JOIN role_permissions rp ON rp.permission_id = p.id AND rp.is_active = 1 LEFT JOIN roles r ON r.id = rp.role_id GROUP BY p.id,
p.code ORDER BY p.id;
mysql> SELECT COUNT(*) AS parejas_activas FROM role_permissions WHERE is_active = 1;

+---------------------+---------------+----------+------------+----------+
| permiso             | ADMINISTRADOR | CONTADOR | CAPTURISTA | CONSULTA |
+---------------------+---------------+----------+------------+----------+
| company.users.read  | sí            | sí       | no         | sí       |
| company.users.write | sí            | no       | no         | no       |
| accounts.read       | sí            | sí       | sí         | sí       |
| accounts.write      | sí            | sí       | no         | no       |
| concepts.read       | sí            | sí       | sí         | sí       |
| concepts.write      | sí            | sí       | no         | no       |
| transactions.read   | sí            | sí       | sí         | sí       |
| transactions.write  | sí            | sí       | sí         | no       |
| roles.read          | sí            | sí       | no         | no       |
| roles.write         | sí            | no       | no         | no       |
+---------------------+---------------+----------+------------+----------+
+-----------------+
| parejas_activas |
+-----------------+
|              26 |
+-----------------+
```

## 27. MySQL: company_user_roles

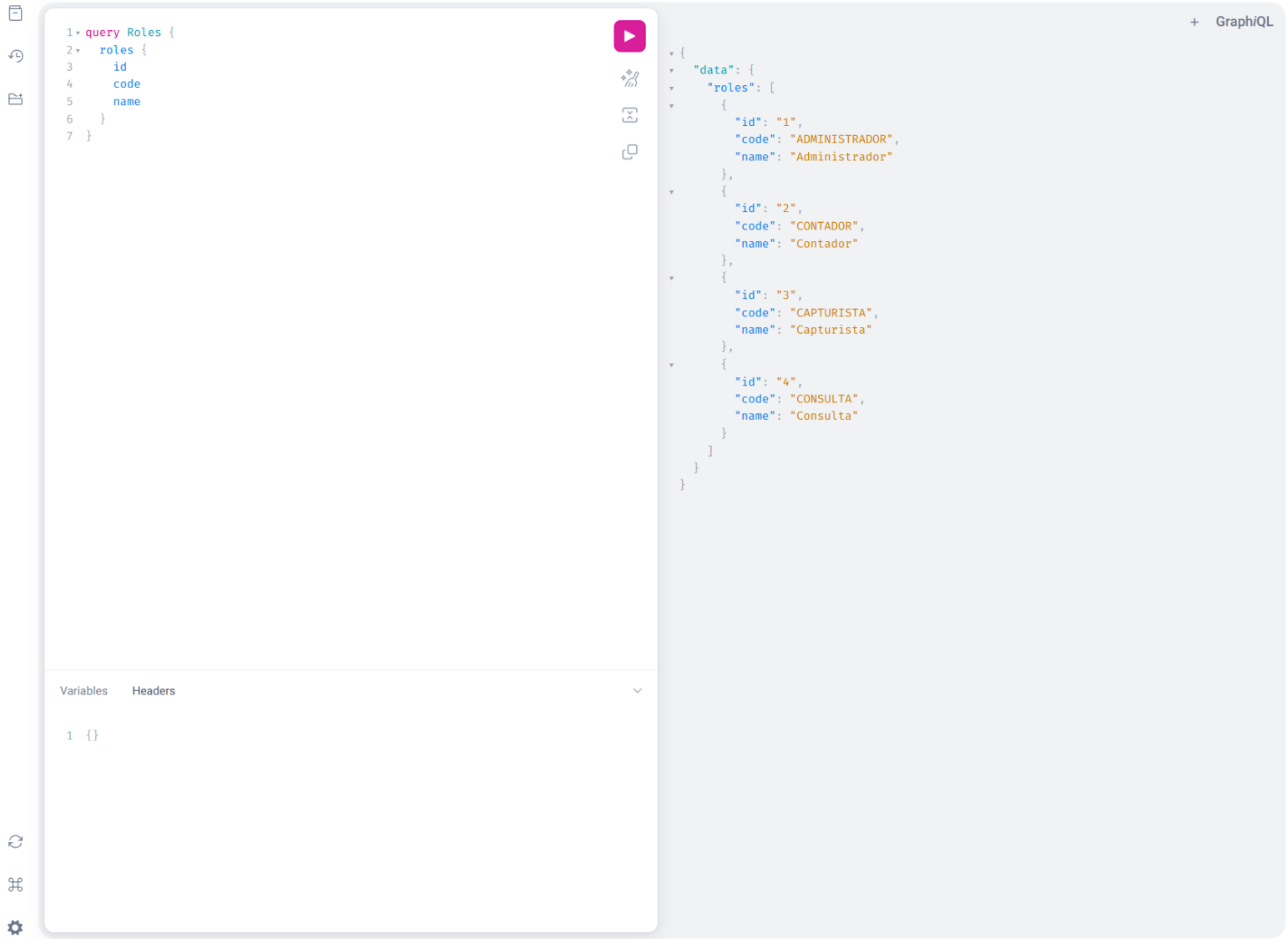
**Regla que demuestra:** Historial de roles por membresía: el rol anterior queda inactivo y cada membresía tiene un solo rol activo.

```
PS C:\Users\Gema\Desktop\Actividad-5> docker exec -it ingresos-mysql mysql -uingresos -p ingresos
mysql> SELECT cur.id, u.name AS usuario, r.code AS rol, cur.is_active FROM company_user_roles cur JOIN company_users cu ON cu.id = cur.company_user_id JOIN users u ON u.id = cu.user_id JOIN roles r ON
r.id = cur.role_id ORDER BY cur.id;

+----+-----------------------+---------------+-----------+
| id | usuario               | rol           | is_active |
+----+-----------------------+---------------+-----------+
|  1 | María Fernanda Gómez  | ADMINISTRADOR |         1 |
|  2 | Luis Herrera          | CONSULTA      |         0 |
|  3 | Luis Herrera          | CONTADOR      |         1 |
|  4 | Ana López             | CONSULTA      |         0 |
|  5 | Ana López             | CAPTURISTA    |         0 |
|  6 | Jorge Ramírez         | CONSULTA      |         1 |
|  7 | Ana López             | CONTADOR      |         1 |
+----+-----------------------+---------------+-----------+
```

## 28. Roles después de reiniciar el servidor

**Regla que demuestra:** Tras reiniciar, la carga inicial solo inserta cuando count() es 0: siguen siendo cuatro roles.



## 29. MySQL: conteos después de reiniciar

**Regla que demuestra:** Los catálogos no se duplicaron: 4 roles, 10 permisos y 26 parejas en role_permissions.

```
PS C:\Users\Gema\Desktop\Actividad-5> docker exec -it ingresos-mysql mysql -uingresos -p ingresos
mysql> SELECT (SELECT COUNT(*) FROM roles) AS roles, (SELECT COUNT(*) FROM permissions) AS permisos, (SELECT COUNT(*) FROM role_permissions) AS role_permissions;

+-------+----------+------------------+
| roles | permisos | role_permissions |
+-------+----------+------------------+
|     4 |       10 |               26 |
+-------+----------+------------------+
```